A

Mini Project Report

On

CLASSIFICATION OF PESTS USING COMPUTER VISION CNN ALGORITHM

Submitted in partial fulfillment of the requirements for the award of Degree

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (AI&ML)

by

K. VENU GOPALA KRISHNA (237R1A6627)
G. SRI LAKSHMI (237R1A6623)
K. ABHINAY (237R1A6628)

Under the Guidance of

Mr. M. RAVINDRAN

Assistant Professor CSE(AIML)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi) Recognized Under Section 2(f) & 12(B) of the UGCAct.1956, Kandlakoya (V), Medchal Road, Hyderabad-501401.

2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

CERTIFICATE

This is to certify that the project entitled **"CLASSIFICATION OF PESTS USING COMPUTER VISION CNN ALGORITHM "** being submitted by **K. VENU GOPALA KRISHNA (237R1A6627), G. SRI LAKSHMI (237R1A6623), K. ABHINAY (237R1A6628)** in partial fulfillment of the requirements for the award of the degree of B.Tech in Computer Science and Engineering (AI&ML) to the Jawaharlal Nehru Technological University Hyderabad, is a record of bonafide work carried out by them under our guidance and supervision during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Mr. M. Ravindran
Assistant Professor CSE(AIML)
INTERNAL GUIDE

Dr. S Rao Chintalapudi
HOD CSE(AI&ML)

EXTERNAL EXAMINER

Submitted for viva voice Examination held on _____

ACKNOWLEDGEMENT

Apart from the efforts of us, the success of any project depends largely on the encouragement and guidelines of many others. We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project.

We take this opportunity to express my profound gratitude and deep regard to my guide **Mr. M. Ravindran,** Assistant Professor for his exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by him shall carry us a long way in the journey of life on which we are about to embark.

We also take this opportunity to express a deep sense of gratitude to the Project Review Committee (PRC) **Mr. M. Balaji, Dr. G. Vinoda Reddy, Dr.P.Harikrishna, Dr. K. Mahesh, Dr.V.Malsoru, Dr.Md.Shareef, Mr M. Ravindran, Mr.G M.Subhani** for their cordial support, valuable information and guidance, which helped us in completing this task through various stages.

We are also thankful to **Dr. S Rao Chintalapudi,** Head, Department of Computer Science and Engineering (AI&ML) for providing encouragement and support for completing this project successfully.

We are obliged to **Dr. A. Raji Reddy,** Director for being cooperative throughout the course of this project. We also express our sincere gratitude to Sri. **Ch. Gopal Reddy,** Chairman for providing excellent infrastructure and a nice atmosphere throughout the course of this project.

The guidance and support received from all the members of **CMR Technical Campus** who contributed to the completion of the project. We are grateful for their constant support and help.

Finally, we would like to take this opportunity to thank our family for their constant encouragement, without which this assignment would not be completed. We sincerely acknowledge and thank all those who gave support directly and indirectly in the completion of this project.

K. VENU GOPALA KRISHNA (237R1A6627)
G. SRI LAKSHMI (237R1A6623)
K. ABHINAY (237R1A6628)

ABSTRACT

The primary strategy for mitigating lost productivity entails promptly, accurately, and efficiently detecting plant pests. Although detection by humans can be useful in detecting certain pests, it is often slower compared to automated methods, such as machine learning. Hence, this study employs a Convolutional Neural Network (CNN) model, specifically Google Net, to detect pests within mobile applications. The technique of detection involves the input of images depicting plant pests, which are subsequently subjected to further processing. This study employed many experimental methods to determine the most effective model. The model exhibiting a 93.78% accuracy stands out as the most superior model within the scope of this investigation. The aforementioned model has been included in a smartphone application with the purpose of facilitating Indonesian farmers in the identification of pests affecting their crops. The implementation of an Indonesian language application is a contribution to this research. Using this local language makes it easier for Indonesian farmers to use it. The potential impact of this application on Indonesian farmers is anticipated to be significant. By enhancing pest identification capabilities, farmers may employ more suitable pest management strategies, leading to improved crop yields in the long run.

Keywords: pest detection; Google Net; Convolutional Neural Network; mobile application

LIST OF FIGURES

Figure No	Figure Name	Page No
Figure 3.3.1	Process Model for Classification of Pests	13
Figure 3.3.2	Requirements Gathering stage for Classification of Pests	14
Figure 3.3.3	Analysis Stage for Classification of Pests	16
Figure 3.3.4	Design Stage for Classification of Pests	17
Figure 3.3.5	Development Stage for Helmet Detection	18
Figure 3.3.6	Integration & Testing Stage for Classification of Pests	19
Figure 3.3.7	Installation & Acceptance Stage for Classification of Pests	20
Figure 4.1	Class Diagram for Classification of Pests	24
Figure 4.2	Use Case Diagram for Classification of Pests	25
Figure 4.3	Sequence Diagram for Classification of Pests	26
Figure 4.4	Collaboration Diagram for Classification of Pests	27
Figure 4.5	Component Diagram for Classification of Pests	28
Figure 4.6	Deployment Diagram for Classification of Pests	29
Figure 4.7	Activity Diagram for Classification of Pests	30
Figure 4.8	Data Flow Diagram for Classification of Pests	31
Figure 5.7.1	Convolutional Neural Network (CNN) Architecture	45
Figure 5.7.2	Image Representation in Terms of Dimensions and Color Channels	46
Figure 5.7.3	Patch Antenna Geometry and Coordinate Representation	46
Figure 5.7.4	Max Pooling Operation in Convolutional Neural Networks	48

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
6.1	TEST CASES	59

TABLE OF CONTENTS

ABSTRACT i

LIST OF FIGURES ii

LIST OF TABLES iii

1. INTRODUCTION 1

    1.1 OBJECTIVE OF THE PROJECT 4

2. LITERATURE SURVEY 5

3. SYSTEM ANALYSIS 12

    3.1 EXISTING SYSTEM 12

    3.2 PROPOSED SYSTEM 12

    3.3 PROCESS MODEL USED WITH JUSTIFICATION 13

    3.4 SOFTWARE REQUIREMENT SPECIFICATION 21

        3.4.1 OVERALL DESCRIPTION 21

        3.4.2 EXTERNAL INTERFACE REQUIREMENTS 23

4. SYSTEM DESIGN 24

5. IMPLEMENTATION 32

    5.1 PYTHON 32

    5.2 INSTALLATION 33

    5.3 PYTHON FEATURES 34

    5.4 PYTHON GUI (TKINTER) 34

    5.5 PYTHON IDLE 35

5.6 LIBRARIES 37

5.7 ALGORITHMS 44

5.8 SAMPLE CODE 48

6. TESTING 57

7. RESULT 60

8. CONCLUSION 64

9. REFERENCES 65

INTRODUCTION

1. INTRODUCTION

Food is one of the most fundamental requirements for maintaining human health and vitality. Almost anywhere in Indonesia, a wide variety of crops may be grown due to the country's abundance of arable land. When it comes to plant exports, Indonesia is a major player. Due to its tropical location, Indonesia frequently experiences floods, droughts, and other forms of severe weather. The changing patterns of precipitation and temperature brought on by global warming also threaten plant growth and crop output. Traditional, inefficient farming practices are still used by some Indonesian farmers. Another factor that might prevent farmers from boosting output is a lack of access to cutting-edge agricultural technologies like organic fertilizer, improved seeds, enough irrigation, and safe pesticides. Social disparities in Indonesian society, which may be experienced by farmers in particular locations, may also have an impact on agricultural output. Small farmers' inability to run their farms efficiently is largely attributable to their lack of knowledge. Poor harvest outcomes are to be expected under these conditions. The situation of farmers in Indonesia is strongly connected to the challenge of identifying pests in agricultural settings, which in turn can have an effect on crop yields that are not at their full potential. The education level of many farmers in Indonesia is rather poor, and they have limited access to agricultural training and knowledge. As a consequence of this, they might not have an appropriate understanding of how to properly detect and control agricultural pests. Smaller farms may not have the financial means to invest in or learn to utilize sophisticated software tools for pest identification. As a result, it may be challenging to see pests in their early stages. Without prompt and proper treatment, a pest infestation on one plant will spread to others in the area. In a typical crop, a single rat may harm anywhere from 11 to 176 plants in a single night. During the breeding season, a single rat can destroy up to 26 stalks in a single night, leaving behind only a few rows of plants. It takes a mouse only 21 days to have her own litter of 6–8 young. Obviously, the severity of this problem might increase dramatically if it is not addressed promptly and effectively. Depending on the surrounding environment and the care offered to the plant, there may be a wide variety of pests that target a single plant type. Several ancillary technologies are already in use in the agriculture industry, including drones. Farmers may utilize drones to keep an eye on crops from above and protect them from pests by spraying pesticides over large areas. However, without special detecting

equipment, drones cannot identify which pests are present on a farm, and not every farmer can afford to buy them. Pest detection apps for smartphones that are already ubiquitous in Indonesia are a necessity. It is important to be able to identify the numerous plant pests that occur. Machine learning using supervised model approaches is one approach. This technique needs data that already have a previous label. In machine learning, there are also various methods of image classification. One effective method is the use of CNNs. The Pest Region-CNN end-to-end detection model, otherwise known as Pest R-CNN, was created by Du et al. for the maize pest Spodoptera frugiperda. It tracks their consumption of maize leaves using the Faster R-CNN model. The model classifies invasion intensity into juvenile, minor, moderate, and severe using UAV-acquired high-spatial-resolution RGB ortho-images. The model demonstrated the efficacy of deep learning object detection in pest monitoring with a mean average accuracy of 43.6% on the test dataset. Their model's detection accuracy improved by 12% and 19% over Faster R-CNN and YOLOv5, respectively. Jiao et al. developed a CNN-based multi-class pest identification approach for complicated scenarios. Their research introduces adaptive feature fusion within feature pyramid networks to extract deeper pest characteristics. To reduce information loss in the highest-level feature map, an adaptive augmentation module was designed. Finally, a two-stage region-based CNN (R-CNN) refined the predicted bounding boxes to determine the pest types and locations in each image. Compared to SSD, Retina Net, FPN, Dynamic R-CNN, and Cascade R-CNN, their method achieved 77.0% accuracy, which `was the highest. The key to lowering the amount of produce that is lost is finding plant pests as early, correctly, and effectively as possible. In this scenario, apps that run on mobile devices provide a significant and pertinent solution. With the help of smartphone applications, farmers are able to conveniently perform routine checks on their crops. They can capture images of plants and detect pests using their mobile devices, making it simpler for them to record and monitor everyday agricultural conditions. Mobile applications have been developed with user interfaces that are straightforward and simple to operate. Because of this, they may be used without trouble by farmers who have varied levels of education and prior experience with technology. A number of applications have been developed to help farmers detect pests. Prabha et al. developed an Android mobile application that utilizes a Deep CNN (DCNN)-based artificial intelligence system for the purpose of insect identification in maize production. The utilization of CNN models on mobile devices yields advantages for all stakeholders, with particular benefits being offered

to farmers and agricultural extension experts, since they enhance their accessibility. A mobile application for Android was developed for the purpose of detecting fall armyworm infection in maize crops. This application incorporated the advice provided by Tamil Nadu Agricultural University's Integrated Pest Management (TNAU IPM) capsules. This mobile software offers guidance on effectively managing the issue of fall armyworm infestation. Chen et al. presented an application for the identification of scale pests in Taiwan using image-based methods. The Coccidia and Diaspididae species, known as mealybugs, are the predominant pests of scale insects in Taiwan. These species have the potential to cause significant harm to plants and pose a substantial threat to agricultural productivity. Therefore, the identification of scale pests holds significant importance within the agricultural sector of Taiwan. You Only Look Once v4 (YOLO v4), single-shot multi-box detectors (SSDs), and faster region-based CNNs (Faster R-CNNs) are some of the deep learning algorithms used to detect and precisely localize scale pests inside an image. A smartphone application has been created that utilizes a trained scale pest detection model. This application aims to assist in the identification of pests in agricultural settings, hence enabling farmers to effectively apply targeted pesticides and minimize crop losses. Despite the existence of several related applications in other countries, there is a distinct necessity for tailored mobile-based applications designed specifically for Indonesian farmers. The compatibility between applications produced in other countries and the local language may not always be guaranteed. The development of a mobile application tailored to Indonesian farmers would facilitate the incorporation of local language into the program's content and user interface. This approach would enhance comprehension and acceptance among farmers, hence facilitating their utilization of the application. There is a poor general level of English proficiency in the whole population of Indonesia. Farmers who have completed elementary and secondary schooling will also find this to be a difficult situation. As a result, developing a mobile application tailored to the needs of Indonesian farmers has the potential to address the distinct obstacles encountered within the agricultural sector in Indonesia. This study also employs a CNN architecture, namely Google Net, for the purpose of identifying pests within mobile applications. Google Net makes use of a technology called Inception blocks, which integrate a variety of convolutional algorithms and filter sizes into a single layer. This results in a reduction in the number of computing processes and parameters that are necessary, making the model simpler. When compared to other CNN architectures that were available

at the time, such as Alex Net or VGG, this model featured a significantly lower total number of parameters. This enabled a more effective utilization of the RAM of the mobile device. In addition, this model includes methods such as L2 regularization and dropout to avoid overfitting. This can assist in retaining model performance on small datasets without the need for a high number of parameters by reducing the likelihood of the model being overly accurate. The fact that Google Net was developed with hardware capabilities in mind, such as the Single Instruction Multiple Data (SIMD) instructions that are present on many recent mobile devices, is another key aspect to take into consideration. This helps to maximize the usage of the resources available in the hardware. Traditional convolution is rendered obsolete as a result of the development of depth-wise separable convolution. Because of this, the number of operations and parameters that are necessary is decreased, which results in increased computing efficiency.

1.1 Objective of the project:

The primary strategy for mitigating lost productivity entails promptly, accurately, and efficiently detecting plant pests. Although detection by humans can be useful in detecting certain pests, it is often slower compared to automated methods, such as machine learning. Hence, this study employs a Convolutional Neural Network (CNN) model, specifically Google Net, to detect pests within mobile applications. The technique of detection involves the input of images depicting plant pests, which are subsequently subjected to further processing. This study employed many experimental methods to determine the most effective model. The model exhibiting a 93.78% accuracy stands out as the most superior model within the scope of this investigation. The aforementioned model has been included in a smartphone application with the purpose of facilitating Indonesian farmers in the identification of pests affecting their crops. The implementation of an Indonesian language application is a contribution to this research. Using this local language makes it easier for Indonesian farmers to use it. The potential impact of this application on Indonesian farmers is anticipated to be significant. By enhancing pest identification capabilities, farmers may employ more suitable pest management strategies, leading to improved crop yields in the long run.

Keywords: pest detection; Google Net; Convolutional Neural Network; mobile application

LITERATURE SURVEY

2. LITERATURESURVEY

‘‘Disaster mitigation strategies,’’

Indonesia experiences various disasters that affect human life in the economic and social aspects. Those disasters also affect the agriculture sector, and farmers have to face destroyed land and crop loss. The lives of farmers become more vulnerable. This paper presents the various disasters in Indonesia that affect agriculture, how the farmers’ lives get affected, and how the farmers cope with the impacts of the disasters with various mitigation strategies. Management of disaster risk reduction is being conducted by farmers with their local wisdom such as using predicting climate change with pranata mangsa in Java, and applying pikukuh for earthquake resistant house construction in Badduy Tribe. It is supported by the relevant stakeholders that, time by time, gets more support with advanced sciences. Farmers do have their own ways to cope with disasters, but other parties’ involvement is needed to make disaster risk reduction management more effective such as through education or direct and indirect aids. Farmers cannot work along for the disaster mitigation, therefore good inter stakeholder coordination is needed.

‘‘Sensing versus the area sampling frame method in paddy rice acreage estimation in Indramayu regency,’’

With a population of 267 million, Indonesia faces the significant challenge of inaccurate rice production data leading to a flawed national rice import policy and supply problems. Its 2018 rice production and harvest area data were generated through the Area Sampling Frame (ASF) method which incurred high labour and financial costs as well as failing to optimize accuracy; hence, an alternative method needs exploration. This study compares ASF and remote sensing Synthetic Aperture Radar (SAR) methods to calculate rice growth stages (RGS) using Indramayu Regency, the highest rice producer in West Java Province, as the study area. The SAR-based method used time-series of Vertical Horizontal (VH) polarization of Sentinel-1A data that employed a combination of k-means clustering, hierarchical cluster analysis (HCA), a visual interpretation and support vector machine (SVM) classifier. Both SAR and ASF methods can generate results on a monthly basis, although remote sensing satellite time revisits can be shortened (every 12 days). Whilst the

ASF, a basic technique for collecting agricultural statistics, was easy to implement in large-scale areas its accuracy depended on the quantity and representativeness of the samples. This study applied the ASF by simulating a sample size of 1.7%, 3.3% and 5% of a rice field area with unmanned aerial vehicles (UAVs) data as a reference. Whilst remote sensing SAR methods involve complex data processing the image classification process can be conducted automatically and cost-effectively (data and its software are free of charge). Moreover, it yields not only statistical data on RGS but also determines the spatial planting patterns and the RGS distribution at 10 m pixel resolution. This method showed more accurate results with overall accuracy of image classification of 81.89% and a kappa coefficient (κ) of 0.73. The comparative result was relatively small, i.e., 4,094.89 ha more than the ASF results (3.5% difference), since this study covered a limited research area. Nonetheless, with evidence of more accurate results remote sensing holds the potential for replication across the country's 416 regencies, so enabling government to develop more appropriate policies that minimize the risks of either a surplus or shortage in the nation's most important food supply.

''Management of diseases and pests in crops by vermicompost and vermicompost tea,''

The intensive use of inorganic fertilizers and pesticides in the agricultural field has globally destroyed soil fertility, killed beneficial microorganisms, and also decreased natural resistance in crops, thereby making them more vulnerable to diseases besides affecting human health and the environment. To overcome these problems, it is very important to shift our attention towards eco-friendly alternatives like vermicompost and vermicompost tea which not only can increase crop growth and yield, suppress diseases and pests sustainably but can also protect human health and the environment. Vermicompost with its rich nutrient content, plant growth promoters like auxins, gibberellins, cytokinins, and beneficial microbes not only improves the growth and yield of crops but also increases the diversity and activity of antagonistic microbes and nematodes, which helps to suppress pests and diseases caused by soil-borne phytopathogens. Vermicompost tea also has a tremendous potential to protect plants from diseases and its application to plants can coat leaf surfaces and reduce available sites for pathogen infection or increases microbial diversity that can kill harmful pathogens. Here, we review recent scientific achievements towards the management of crop diseases and pests by these organic amendments and the major points are the following: (1) production of vermicompost and vermicompost tea, (2) management

of crop pests and diseases by vermicompost and vermicompost tea, and (3) the possible mechanisms and some important factors involved in the suppression of diseases and pests. Finally, we conclude that by using these eco-friendly organic amendments as a replacement to inorganic pesticides and fungicides, diseases and pests can be managed successfully without affecting human health and the environment and chemical-free food can be provided to humankind in the future.

''The Impact of Credit on Multidimensional Poverty in Rural Areas,''

Poor farmer households are a vulnerable group in rural areas. Various poverty alleviation measures have been launched to help poor farmer groups become more prosperous. The policies launched were in the form of cash transfers, empowerment, and access to formal financial institutions. Policies for providing formal financial access continue to face many obstacles, one of which is credit risk and farmer literacy. The study aims to estimate the impact of credit on multidimensional poverty in poor farmer households in Indonesia. Secondary data were obtained from the Indonesian Family Life Survey (IFLS) batch 4 and 5. The impact estimation method used was propensity score matching combined with the difference in differences. The results showed that credit programs for poor farmers, initiated by official financial institutions, significantly helped farmers out of poverty, although the value was small. The addition of control variables such as education, ownership of household assets, and ownership of agricultural land actually made the credit program more modifiable at the policy level. Credit can be used as a complement to policies related to improving farmer education and knowledge in the form of agricultural modernization, as well as to scale-up of farmer household businesses.

''The Adaptation of Export-Scale Urban Farmers Amid the COVID-19 Pandemic in Bandung Metropolitan,''

These days, urban agriculture is more than a hobby. It has expanded into a local commercial business, even to an export scale. However, urban farmers who have commercialized their products must adapt to the COVID-19 pandemic situation, which has impacted many aspects of global life. This research used a mixed-method approach. We collected quantitative data from 107 respondents on the household commercialization index, income level, and education level of export-scale-urban farmers in the Bandung metropolitan area, West Java, Indonesia. We also used qualitative data to determine how farmers were

adapting to difficult situations due to the COVID-19 pandemic. This information was gathered through in-depth interviews, observation, and documentation. The results showed that the urban farmers continued to prioritize commercial agriculture during the COVID-19 pandemic. The urban farmers must adapt to the COVID-19 pandemic in various ways, beginning with market access, agricultural management, and strengthening financial resources. There are some conditions where adaptation methods adopted by farmers are not environmentally friendly because the farmers are increasingly dependent on synthetic inputs and use cold storage on open land. Thus, the farmers' adaptation steps to maintain export-scale commercial farming activities in the situation of the COVID-19 pandemic also affect intensive agricultural practices that are not environmentally friendly.

"The first report on the prevalence of soil transmitted helminth infections and associated risk factors among traditional pig farmers,"

Pigs are the main livestock commodity in Bali Province, Indonesia, where traditional farming practices are widely used. Traditional pig farmers are often closely associated with poverty and a perceived lack of knowledge regarding health and hygiene. Data on soil-transmitted helminthiasis (STH) and risk factors associated with STH worm infection among traditional pig farmers in Bali were previously unavailable. This study aimed to analyze the prevalence and risk factors for STH infections among traditional pig farmers in Bali Province, Indonesia. Materials and methods: This study involved the fecal examination of 238 traditional pig farmers from all areas of Bali Province in Indonesia. In addition, several pig feces samples were combined into one pooled sample belonging to each farm. All fresh fecal samples were stored in a 5% formaldehyde solution before being analyzed using concentration flotation techniques. Subsequently, risk factors were determined through an interview and a questionnaire. The odds ratio (OR) and Chi-square tests were used to determine the risk factors associated with STH infections. Results: The result showed that there was a high prevalence of STH infections among traditional pig farmers and pig farms in Bali, with rates of 21.8% and 76.5%, respectively. This could be due to risk factors, such as personal hygiene confidence interval, sanitation, education level, household income, and occupation. Conclusion: The infections seen in farmers were distributed among hookworm, Ascaris spp., and Trichuris spp., at 15.1%, 9.2%, and 4.2%, respectively. The risk factors associated with infections of STH and Ascaris spp. were personal hygiene, home sanitation, education level, household income, and having a primary occupation as a traditional pig

farmer. In contrast, personal hygiene, education level, and primary occupation were the only risk factors for hookworm infection, while personal hygiene and home sanitation were the risk factors associated with Trichuris spp. infection. The limitation of this study was that the number of samples was relatively small due to the difficulty of obtaining stool samples from traditional pig farmers, with many individuals refusing to provide their stool for inspection. We suggest that future research focus on identifying the species of worms that infect traditional pig farmers and to better identify the zoonotic link of STH transmission from pigs to humans.

"Pest Early Detection in Greenhouse Using Machine Learning,"

Greenhouses are considered to be a favorable artificial environment separated from the outside. However, pests can still exist by the same plant sources that bring the pathogen. The conditions and abundant food in a greenhouse provide a stable environment for the pest development. Normally, the natural enemies that serve to keep pests under control outside are not present in the greenhouse, pest situations often develop in this indoor environment more rapidly and with greater severity than outdoors. Early detection and diagnosis of pests and diseases are key to managing greenhouse pests as well as selecting and applying appropriate pesticides when needed. The aim of this invention is to develop an intelligent pest early detection system using a convolutional neural network in the greenhouse. By using a pre-trained disease recognition model, we were able to perform deep transfer learning to produce a network that can predict with the precision above 90%.

"Real-time detection algorithm of tomato diseases and pests in the natural environment. Plant Methods,"

Background Research on early object detection methods of crop diseases and pests in the natural environment has been an important research direction in the fields of computer vision, complex image processing and machine learning. Because of the complexity of the early images of tomato diseases and pests in the natural environment, the traditional methods cannot achieve real-time and accurate detection. Results Aiming at the complex background of early period of tomato diseases and pests image objects in the natural environment, an improved object detection algorithm based on YOLOv3 for early real-time detection of tomato diseases and pests was proposed. Firstly, aiming at the complex background of tomato diseases and pests images under natural conditions, dilated convolution layer is used

to replace convolution layer in backbone network to maintain high resolution and receptive field and improve the ability of small object detection. Secondly, in the detection network, according to the size of candidate box intersection ratio (IOU) and linear attenuation confidence score predicted by multiple grids, the obscured objects of tomato diseases and pests are retained, and the detection problem of mutual obscure objects of tomato diseases and pests is solved. Thirdly, to reduce the model volume and reduce the model parameters, the network is lightweight by using the idea of convolution factorization. Finally, by introducing a balance factor, the small object weight in the loss function is optimized. The test results of nine common tomato diseases and pests under six different background conditions are statistically analyzed. The proposed method has a F1 value of 94.77%, an AP value of 91.81%, a false detection rate of only 2.1%, and a detection time of only 55 Ms. The test results show that the method is suitable for early detection of tomato diseases and pests using large-scale video images collected by the agricultural Internet of Things. Conclusions At present, most of the object detection of diseases and pests based on computer vision needs to be carried out in a specific environment (such as picking the leaves of diseases and pests and placing them in the environment with light supplement equipment, so as to achieve the best environment). For the images taken by the Internet of things monitoring camera in the field, due to various factors such as light intensity, weather change, etc., the images are very different, the existing methods cannot work reliably. The proposed method has been applied to the actual tomato production scenarios, showing good detection performance. The experimental results show that the method in this study improves the detection effect of small objects and leaves occlusion, and the recognition effect under different background conditions is better than the existing object detection algorithms. The results show that the method is feasible to detect tomato diseases and pests in the natural environment.

"Using citizen science for early detection of tree pests and diseases: Perceptions of professional and public participants,"

Early detection of new tree pests and diseases is a vital element of national strategies to prevent their establishment and spread into a country or region, based on the rationale that it increases the chances of successful eradication. Given the limited capacity and financial resources of most national plant protection authorities, the use of public participants has recently been explored in a range of citizen science projects for its ability to supplement

official surveillance. However, little is known about the motivations, expectations and experiences of members of the public involved in such activities and even less about the views of professionals and officials. In this study, evidence was obtained from structured interviews with professionals and volunteers engaged in five projects related to tree health surveillance. Some differences were noted between the two groups with a greater focus on personal aspects by members of the public and on strategic and institutional aspects by professionals. A striking feature was the agreement of the two groups that the projects had met or exceeded their expectations, and provided the proof of concept that tree health surveillance capacity can be increased by engaging and training citizens. Many participants shared concerns about the importance of securing both project longevity and volunteer participation over the long term. The paper discusses ways in which the motivations of tree health surveillance participants can be sustained over long periods with particular attention to recognition and reward.

"Ethical dilemmas when using citizen science for early detection of invasive tree pests and diseases. Manage,"

The early detection of tree health pests and disease is an important component of biosecurity to protect the aesthetic, recreational and economic importance of trees, woodlands and forestry. Citizen science is valuable in supporting the early detection of tree pests and diseases. Different stakeholders (government, business, society and individual) will vary in their opinion of the balance between costs and benefits of early detection and consequent management, partly because many costs are local whereas benefits are felt at larger scales. This can create clashes in motivations of those involved in citizen science, thus leading to ethical dilemmas about what is good and responsible conduct for the use of citizen science. We draw on our experience of tree health citizen science to exemplify five dilemmas. These dilemmas arise because: the consequences of detection may locally be severe (e.g. the destruction of trees); knowledge of these impacts could lead to refusal to make citizen science reports; citizen science reports can be made freely, but can be costly to respond to; participants may expect solutions even if these are not possible; and early detection is (by definition) a rare event. Effective engagement and dialogue across stakeholders, including public stakeholders, is important to properly address these issues. This is vital to ensure the public's long-term support for and trust in the use of citizen science for the early detection of tree pests and diseases.

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 Existing System

In Traditionally Farmers often lose crops to pests because they can't identify the type of pest and, therefore, can't choose the right pesticide. Different pests need different pesticides, and without proper identification, farmers struggle to control the pests effectively So some existing authors using ML algorithms to implement this project but it is giving less Accuracy and time taking process for prediction.

Disadvantages:

1. High Costs
2. Low Accuracy
3. Time taking

3.2 Proposed System

In propose we are using a Convolutional Neural Network (CNN)-based AI system to help farmers identify pests. By uploading a pest image, the CNN will analyze and classify the pest type, enabling farmers to purchase the correct pesticide and improve pest control and crop production.

Advantages:

1. High Accuracy
2. Cost-Effective
3. Less time taking

Modules Description:

To implement this project, we have designed following modules

1) Train Pest Classification CNN Model: using this module user can train and load CNN model and then perform prediction on 20% test images to calculate prediction accuracy

2) Predict Pest Type using Test Image: using this module user can upload test image and then DJANGO will read image and input to CNN to get classification result

3.3. PROCESS MODEL USED WITH JUSTIFICATION

SDLC (Umbrella Model):

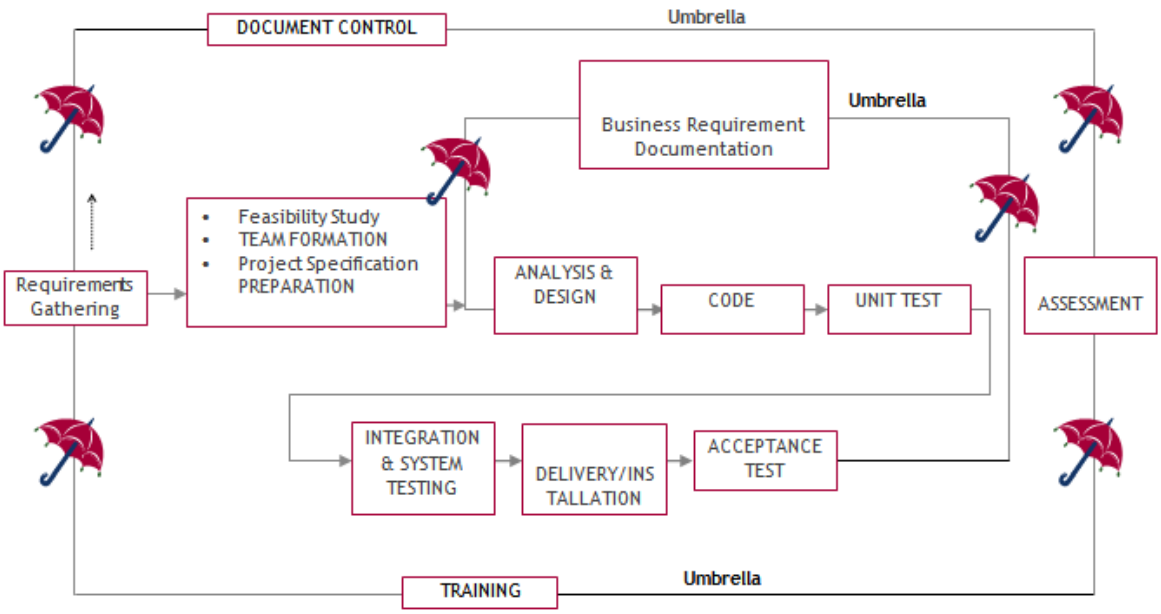

Fig 3.3.1 Process Model for Classification of Pests.

SDLC is nothing but Software Development Life Cycle. It is a standard which is used by software industry to develop good software.

Stages in SDLC:

- Requirement Gathering
- Analysis
- Designing
- Coding
- Testing

- Maintenance

Requirements Gathering stage:

The requirements gathering process takes as its input the goals identified in the high-level requirements section of the project plan. Each goal will be refined into a set of one or more requirements. These requirements define the major functions of the intended application, define operational data areas and reference data areas, and define the initial data entities. Major functions include critical processes to be managed, as well as mission critical inputs, outputs and reports. A user class hierarchy is developed and associated with these major functions, data areas, and data entities. Each of these definitions is termed a Requirement. Requirements are identified by unique requirement identifiers and, at minimum, contain a requirement title and textual description.

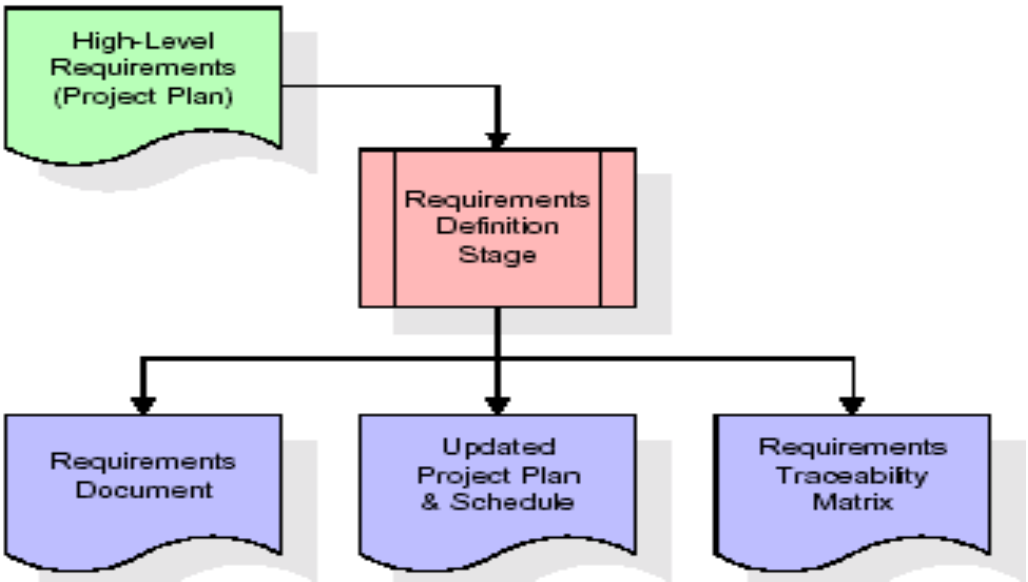

Fig 3.3.2 Requirements Gatheringstage for Classification of Pests.

These requirements are fully described in the primary deliverables for this stage: the Requirements Document and the Requirements Traceability Matrix (RTM). The requirements document contains complete descriptions of each requirement, including diagrams and references to external documents as necessary. Note that detailed listings of database tables and fields are not included in the requirements document.

The title of each requirement is also placed into the first version of the RTM, along with the title of each goal from the project plan. The purpose of the RTM is to show that the product components developed during each stage of the software development lifecycle are formally connected to the components developed in prior stages.

In the requirements stage, the RTM consists of a list of high-level requirements, or goals, by title, with a listing of associated requirements for each goal, listed by requirement title. In this hierarchical listing, the RTM shows that each requirement developed during this stage is formally linked to a specific product goal. In this format, each requirement can be traced to a specific product goal, hence the term requirements traceability.

The outputs of the requirements definition stage include the requirements document, the RTM, and an updated project plan.

- Feasibility study is all about identification of problems in a project.
- No. of staff required to handle a project is represented as Team Formation, in this case only modules are individual tasks will be assigned to employees who are working for that project.
- Project Specifications are all about representing of various possible inputs submitting to the server and corresponding outputs along with reports maintained by administrator.

Analysis Stage:

The planning stage establishes a bird's eye view of the intended software product, and uses this to establish the basic project structure, evaluate feasibility and risks associated with the project, and describe appropriate management and technical approaches.

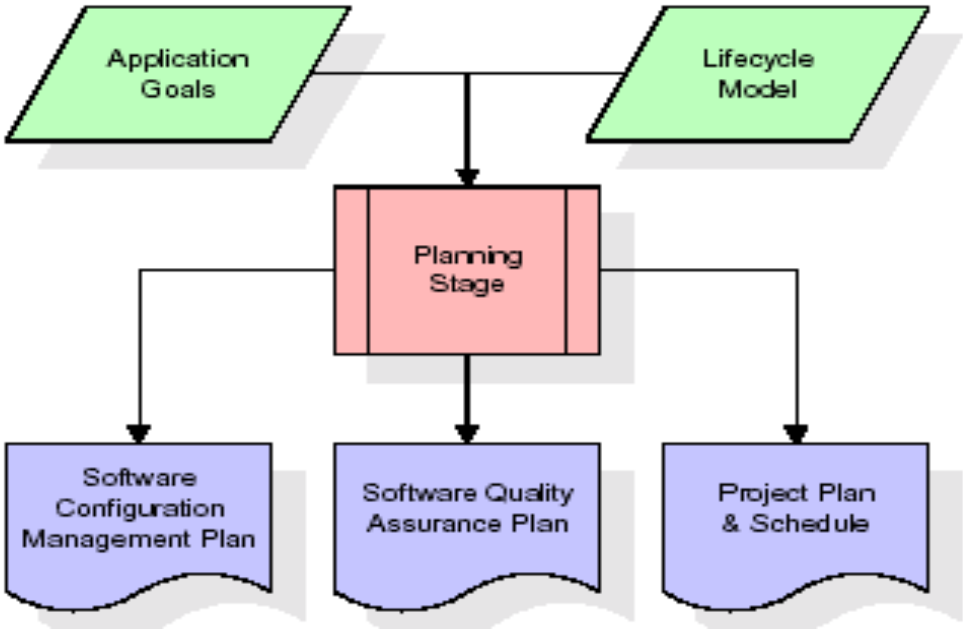

Fig 3.3.3 Analysis Stage for Classification of Pests.

The most critical section of the project plan is a listing of high-level product requirements, also referred to as goals. All of the software product requirements to be developed during the requirements definition stage flow from one or more of these goals. The minimum information for each goal consists of a title and textual description, although additional information and references to external documents may be included. The outputs of the project planning stage are the configuration management plan, the quality assurance plan, and the project plan and schedule, with a detailed listing of scheduled activities for the upcoming Requirements stage, and high-level estimates of effort for the out stages.

Designing Stage:

The design stage takes as its initial input the requirements identified in the approved requirements document. For each requirement, a set of one or more design elements will be produced as a result of interviews, workshops, and/or prototype efforts. Design elements describe the desired software features in detail, and generally include functional hierarchy diagrams, screen layout diagrams, tables of business rules, business process diagrams, pseudo code, and a complete entity-relationship diagram with a full data dictionary. These design elements are intended to describe the software in sufficient detail that skilled programmers may develop the software with minimal additional input.

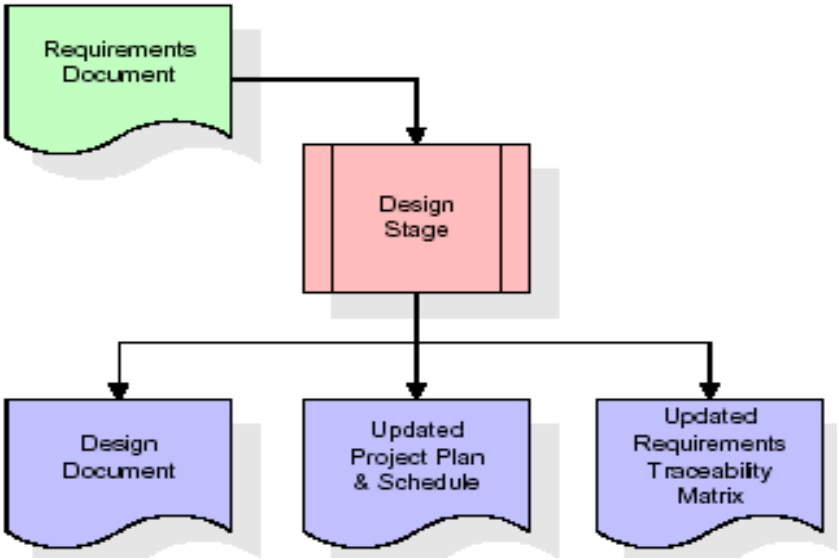

Fig 3.3.4 Design Stage for Classification of Pests.

When the design document is finalized and accepted, the RTM is updated to show that each design element is formally associated with a specific requirement. The outputs of the design stage are the design document, an updated RTM, and an updated project plan.

Development (Coding) Stage:

The development stage takes as its primary input the design elements described in the approved design document. For each design element, a set of one or more software artifacts will be produced. Software artifacts include but are not limited to menus, dialogs, and data management forms, data reporting formats, and specialized procedures and functions. Appropriate test cases will be developed for each set of functionally related software artifacts, and an online help system will be developed to guide users in their interactions with the software.

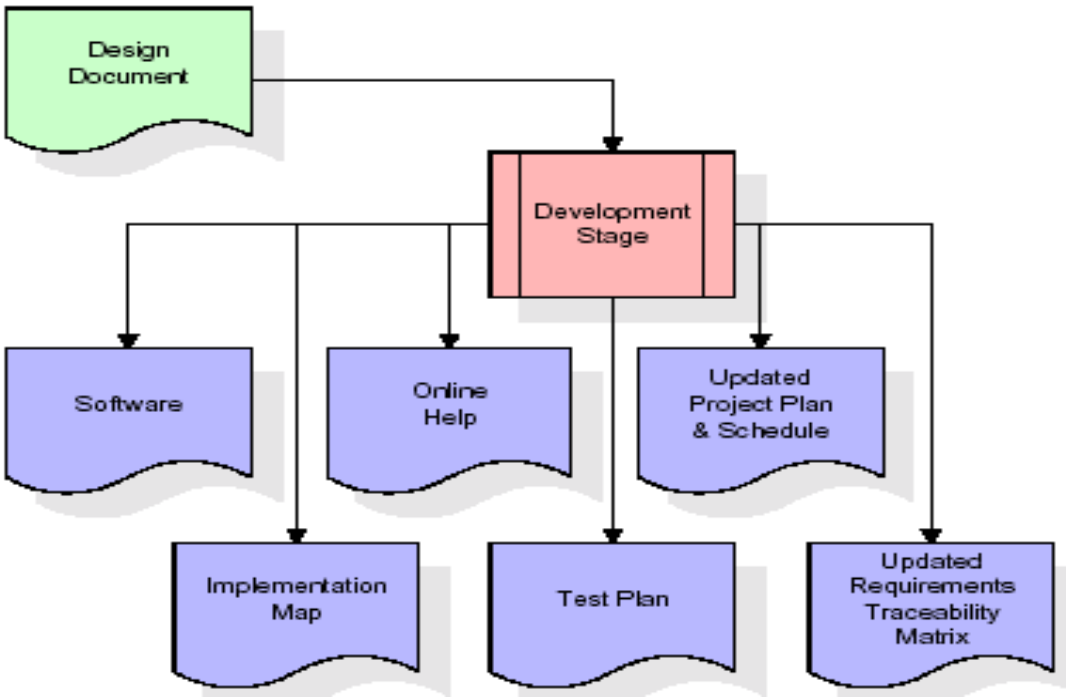

Fig 3.3.5 Development Stage for Classification of Pests.

The RTM will be updated to show that each developed artifact is linked to a specific design element, and that each developed artifact has one or more corresponding test case items. At this point, the RTM is in its final configuration. The outputs of the development stage include a fully functional set of software that satisfies the requirements and design elements previously documented, an online help system that describes the operation of the software, an implementation map that identifies the primary code entry points for all major system functions, a test plan that describes the test cases to be used to validate the correctness and completeness of the software, an updated RTM, and an updated project plan.

Integration & Test Stage:

During the integration and test stage, the software artifacts, online help, and test data are migrated from the development environment to a separate test environment. At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite confirms a robust and complete migration capability. During this stage, reference data is finalized for production use and production users are identified and

linked to their appropriate roles. The final reference data (or links to reference data source files) and production user list are compiled into the Production Initiation Plan.

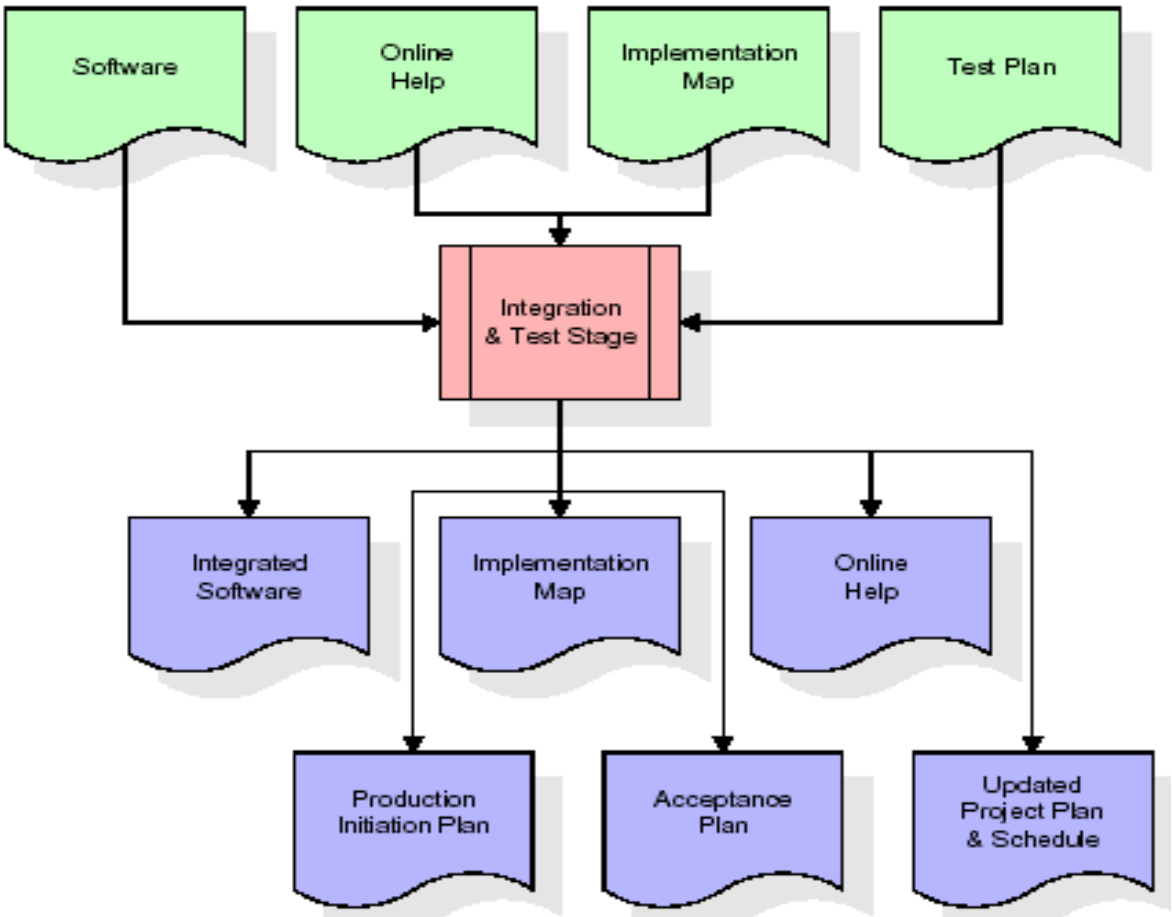

Fig 3.3.6 Integration & Testing Stage for Classification of Pests.

The outputs of the integration and test stage include an integrated set of software, an online help system, an implementation map, a production initiation plan that describes reference data and production users, an acceptance plan which contains the final suite of test cases, and an updated project plan.

- **Installation & Acceptance Test:**

During the installation and acceptance stage, the software artifacts, online help, and initial production data are loaded onto the production server. At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite is a prerequisite to acceptance of the software by the customer.

After customer personnel have verified that the initial production data load is correct and the test suite has been executed with satisfactory results, the customer formally accepts the delivery of the software.

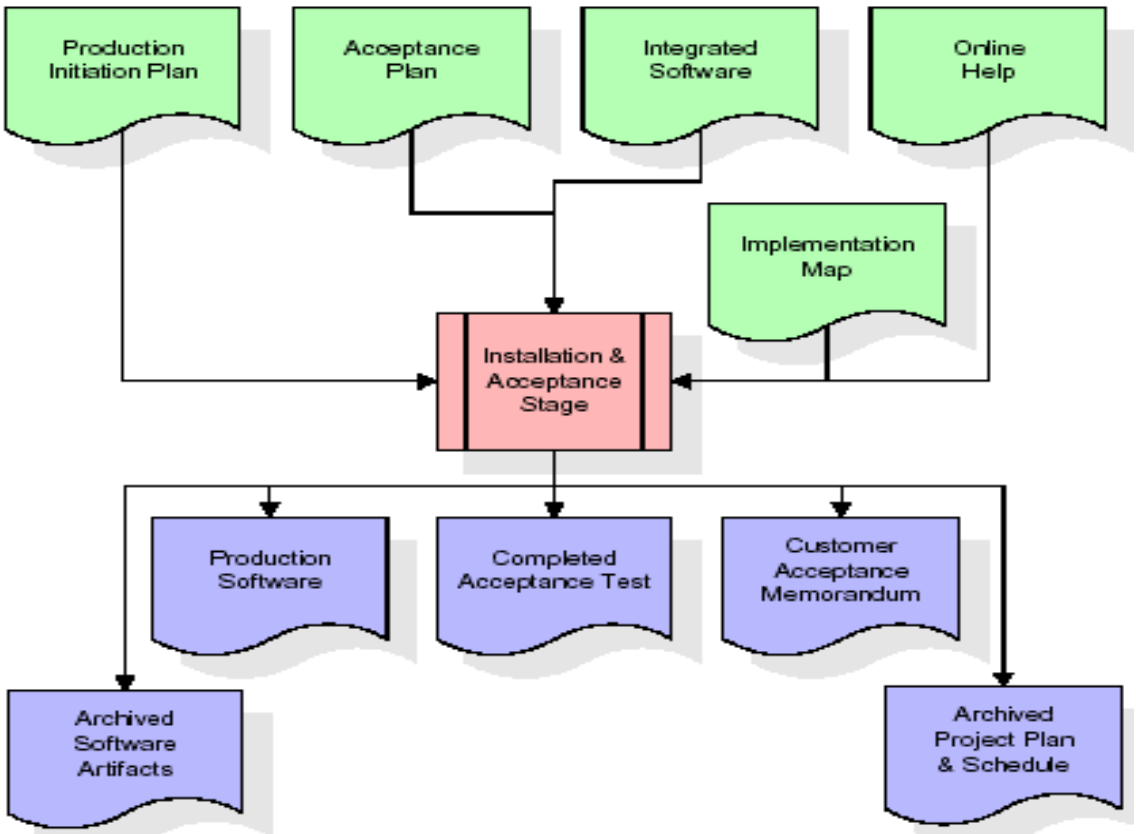

Fig 3.3.7 Installation & Acceptance Stage for Classification of Pests.

The primary outputs of the installation and acceptance stage include a production application, a completed acceptance test suite, and a memorandum of customer acceptance of the software. Finally, the PDR enters the last of the actual labor data into the project schedule and locks the project as a permanent project record. At this point the PDR "locks" the project by archiving all software items, the implementation map, the source code, and the documentation for future reference.

Maintenance:

Outer rectangle represents maintenance of a project, Maintenance team will start with requirement study, understanding of documentation later employees will be assigned work

and they will undergo training on that particular assigned category. For this life cycle there is no end, it will be continued so on like an umbrella (no ending point to umbrella sticks).

3.4. Software Requirement Specification

3.4.1. Overall Description

A Software Requirements Specification (SRS) – a requirements specification for a software system is a complete description of the behaviour of a system to be developed. It includes a set of use cases that describe all the interactions the users will have with the software. In addition to use cases, the SRS also contains non-functional requirements. Non-functional requirements are requirements which impose constraints on the design or implementation (such as performance engineering requirements, quality standards, or design constraints).

System requirements specification: A structured collection of information that embodies the requirements of a system. A business analyst, sometimes titled system analyst, is responsible for analysing the business needs of their clients and stakeholders to help identify business problems and propose solutions. Within the systems development lifecycle domain, the BA typically performs a liaison function between the business side of an enterprise and the information technology department or external service providers. Projects are subject to three sorts of requirements:

- Business requirements describe in business terms what must be delivered or accomplished to provide value.
- Product requirements describe properties of a system or product (which could be one of several ways to accomplish a set of business requirements.)
- Process requirements describe activities performed by the developing organization. For instance, process requirements could specify. Preliminary investigation examine project feasibility, the likelihood the system will be useful to the organization. The main objective of the feasibility study is to test the Technical, Operational and Economical feasibility for adding new modules and debugging old running system. All system is feasible if they are

unlimited resources and infinite time. There are aspects in the feasibility study portion of the preliminary investigation:

- **ECONOMIC FEASIBILITY**

A system can be developed technically and that will be used if installed must still be a good investment for the organization. In the economical feasibility, the development cost in creating the system is evaluated against the ultimate benefit derived from the new systems. Financial benefits must equal or exceed the costs. The system is economically feasible. It does not require any addition hardware or software. Since the interface for this system is developed using the existing resources and technologies available at NIC, There is nominal expenditure and economical feasibility for certain.

- **OPERATIONAL FEASIBILITY**

Proposed projects are beneficial only if they can be turned out into information system. That will meet the organization's operating requirements. Operational feasibility aspects of the project are to be taken as an important part of the project implementation. This system is targeted to be in accordance with the above-mentioned issues. Beforehand, the management issues and user requirements have been taken into consideration. So, there is no question of resistance from the users that can undermine the possible application benefits. The well-planned design would ensure the optimal utilization of the computer resources and would help in the improvement of performance status.

- **TECHNICAL FEASIBILITY**

Earlier no system existed to cater to the needs of 'Secure Infrastructure Implementation System'. The current system developed is technically feasible. It is a web-based user interface for audit workflow at NIC-CSD. Thus, it provides an easy access to. the users. The database's purpose is to create, establish and maintain a workflow among various entities in order to facilitate all concerned users in their various capacities or roles. Permission to the users would be granted based on the roles specified. Therefore, it provides the technical guarantee of accuracy, reliability and security.

3.4.2. External Interface Requirements

User Interface

The user interface of this system is a user-friendly python Graphical User Interface.

Hardware Interfaces

The interaction between the user and the console is achieved through python capabilities.

Software Interfaces

The required software is python.

SYSTEM REQUIREMENT:

HARDWARE REQUIREMENTS:

- Processor - Intel i3(min)
- Speed - 1.1 GHz
- RAM - 4GB (min)
- Hard Disk - 500 GB

SOFTWARE REQUIREMENTS:

- Operating System - Windows10(min)
- Programming Language - Python with Jupiter notebook

SYSTEM DESIGN

4. SYSTEM DESIGN

UML Diagram:

Class Diagram:

The class diagram is the main building block of object-oriented modelling. It is used both for general conceptual modelling of the systematic of the application, and for detailed modelling translating the models into programming code. Class diagrams can also be used for data modelling. The classes in a class diagram represent both the main objects, interactions in the application and the classes to be programmed. In the diagram, classes are represented with boxes which contain three parts:

- The upper part holds the name of the class
- The middle part contains the attributes of the class
- The bottom part gives the methods or operations the class can take or undertake

Class Diagram:

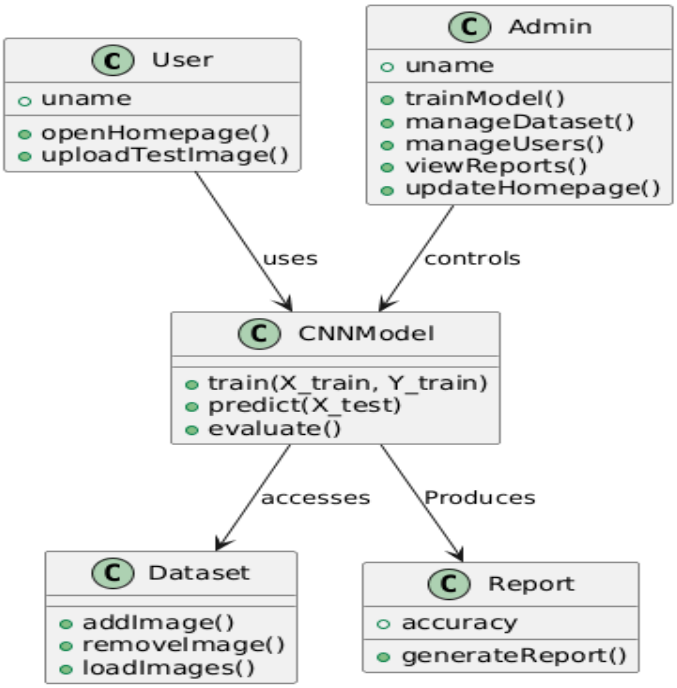

Fig 4.1 Class Diagram for Classification of Pests.

Use case Diagram:

A use case diagram at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system. This type of diagram is typically used in conjunction with the textual use case and will often be accompanied by other types of diagrams as well.

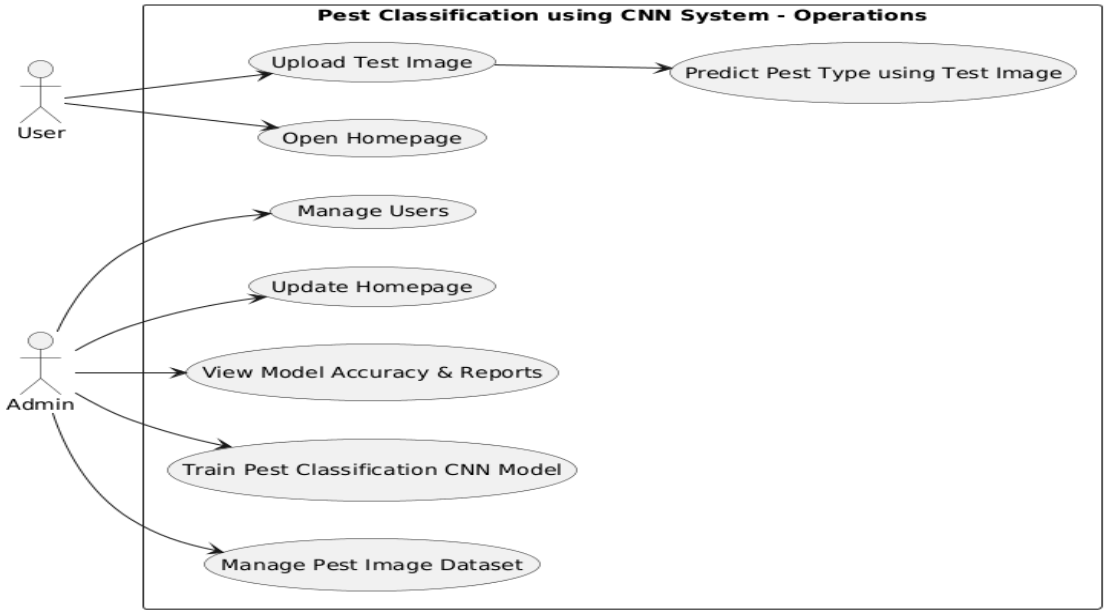

Fig 4.2 Use Case Diagram for Classification of Pests.

Sequence diagram:

A sequence diagram is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called **event diagrams**, **event scenarios**, and timing diagrams.

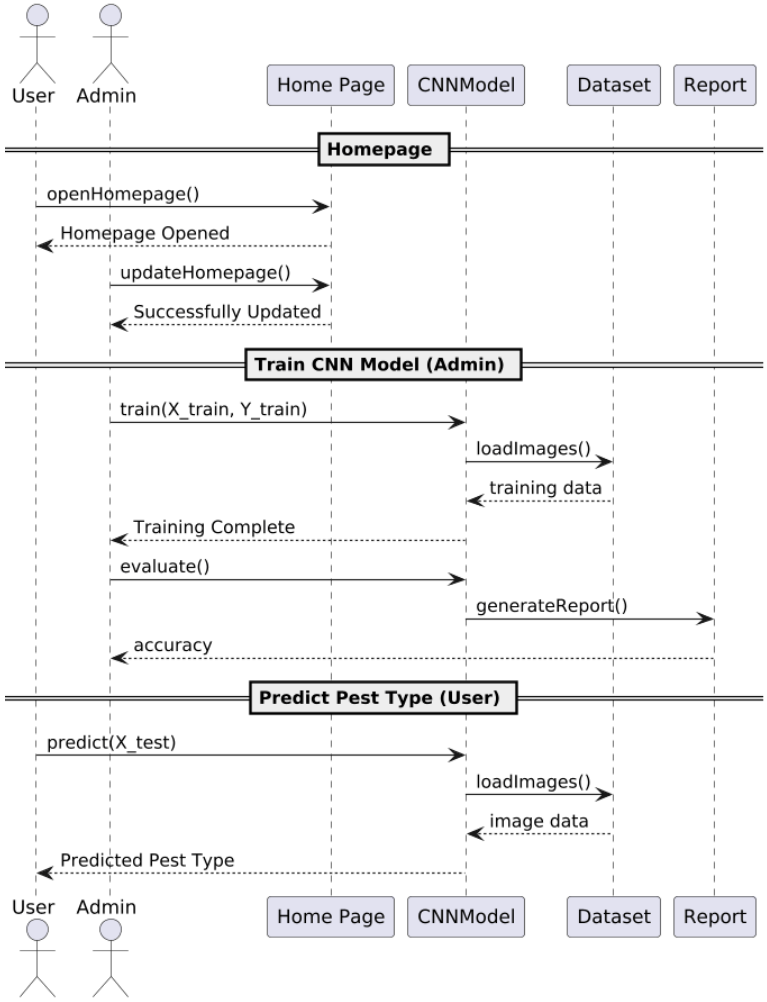

Fig 4.3 Sequence Diagram for Classification of Pests.

Collaboration diagram:

A collaboration diagram describes interactions among objects in terms of sequenced messages. Collaboration diagrams represent a combination of information taken from class, sequence, and use case diagrams describing both the static structure and dynamic behaviour of a system.

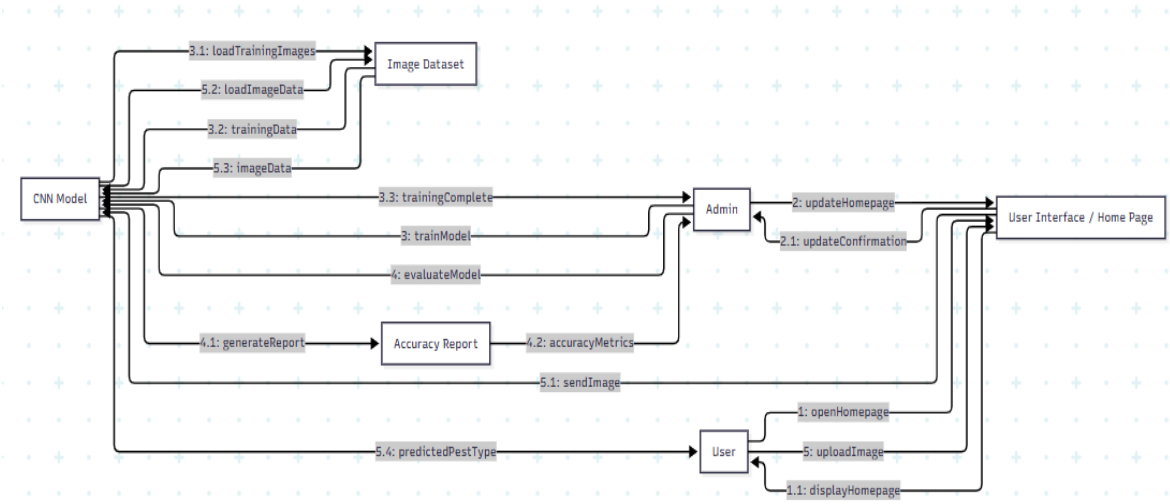

Fig 4.4 Collaboration Diagram for Classification of Pests.

Component Diagram:

In the Unified Modelling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems.

Components are wired together by using an assembly connector to connect the required interface of one component with the provided interface of another component. This illustrates the service consumer - service provider relationship between the two components.

Component Diagram:

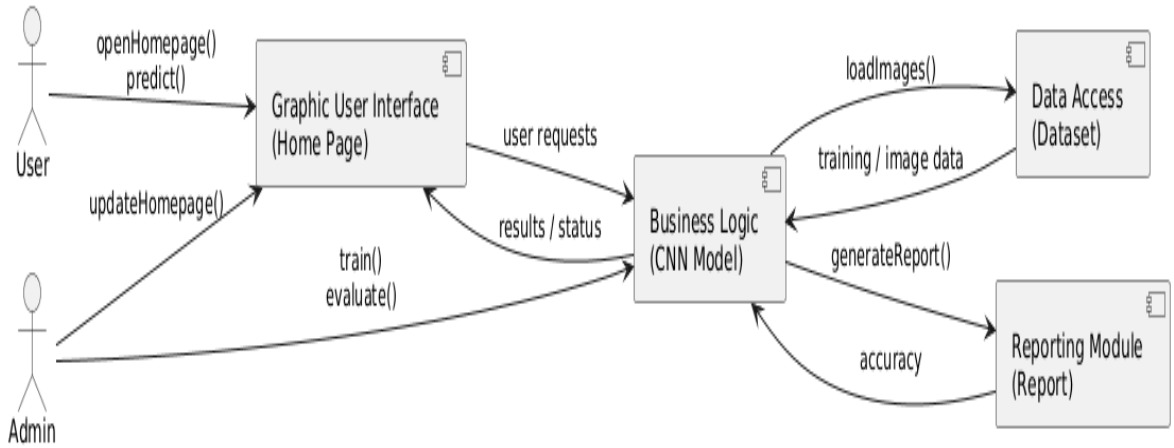

Fig 4.5 Component Diagram for Classification of Pests.

Deployment Diagram:

A **deployment diagram** in the Unified Modelling Language models the physical deployment on nodes. To describe a web site, for example, a deployment diagram would show what hardware components ("nodes") exist (e.g., a web server, an application server, and a database server), what software components run on each node (e.g., web application, database), and how the different pieces are connected (e.g. JDBC, REST, RMI).

The nodes appear as boxes, and the allocated to each node appear as rectangles within the boxes. Nodes may have sub nodes, which appear as nested boxes. A single node in a deployment diagram may conceptually represent multiple physical nodes, such as a cluster of database servers.

Deployment Diagram:

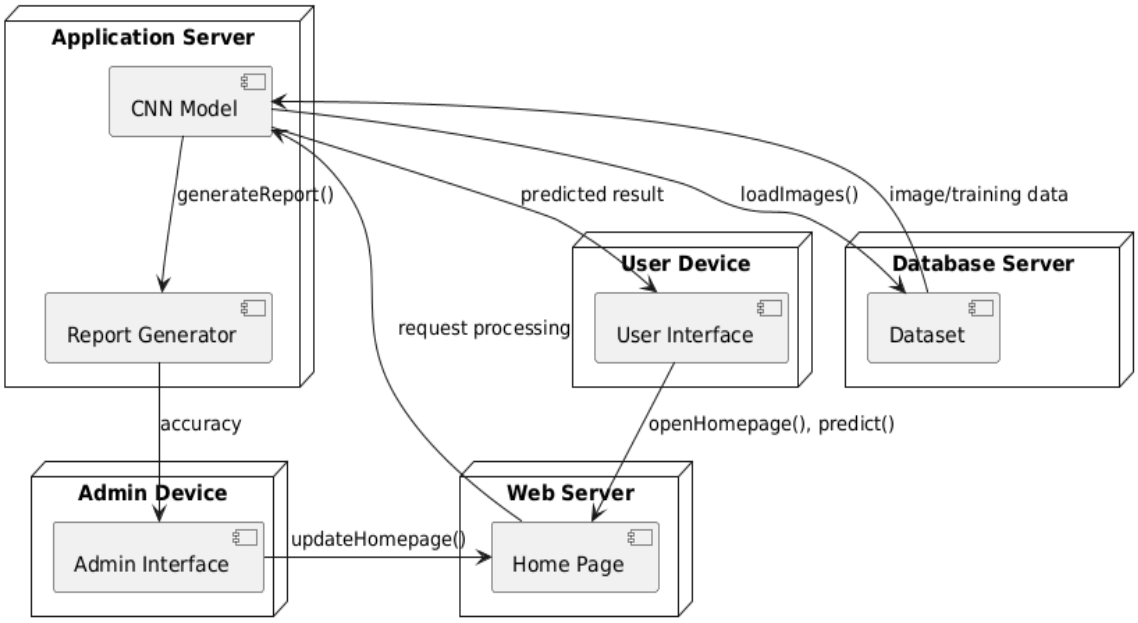

Fig 4.6 Deployment Diagram for Classification of Pests.

Activity Diagram:

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. It is basically a flow chart to represent the flow form one activity to another activity. The activity can be described as an operation of the system. So, the control flow is drawn from one operation to another. This flow can be sequential, branched or concurrent

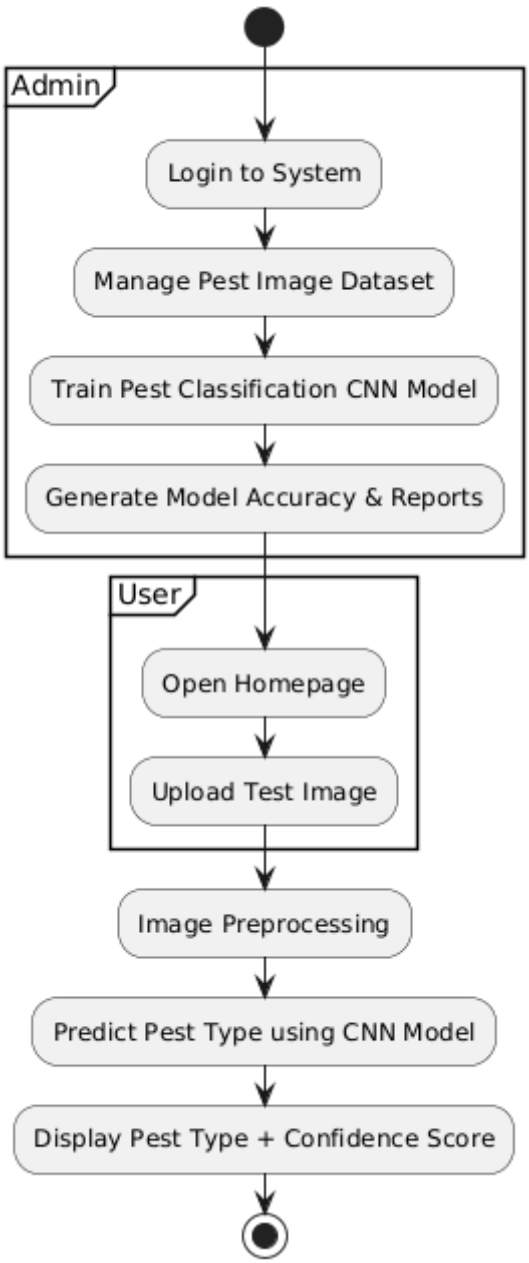

Fig 4.7 Activity Diagram for Classification of Pests.

Data Flow Diagram:

Data flow diagrams illustrate how data is processed by a system in terms of inputs and outputs. Data flow diagrams can be used to provide a clear representation of any business function. The technique starts with an overall picture of the business and continues by analyzing each of the functional areas of interest. This analysis can be carried out in precisely the level of detail required. The technique exploits a method called top-down expansion to conduct the analysis in a targeted way.

As the name suggests, Data Flow Diagram (DFD) is an illustration that explicates the passage of information in a process. A DFD can be easily drawn using simple symbols. Additionally, complicated processes can be easily automated by creating DFDs using easy-to-use, free downloadable diagramming tools. A DFD is a model for constructing and analyzing information processes. DFD illustrates the flow of information in a process depending upon the inputs and outputs. A DFD can also be referred to as a Process Model. A DFD demonstrates business or technical process with the support of the outside data saved, plus the data flowing from the process to another and the end results.

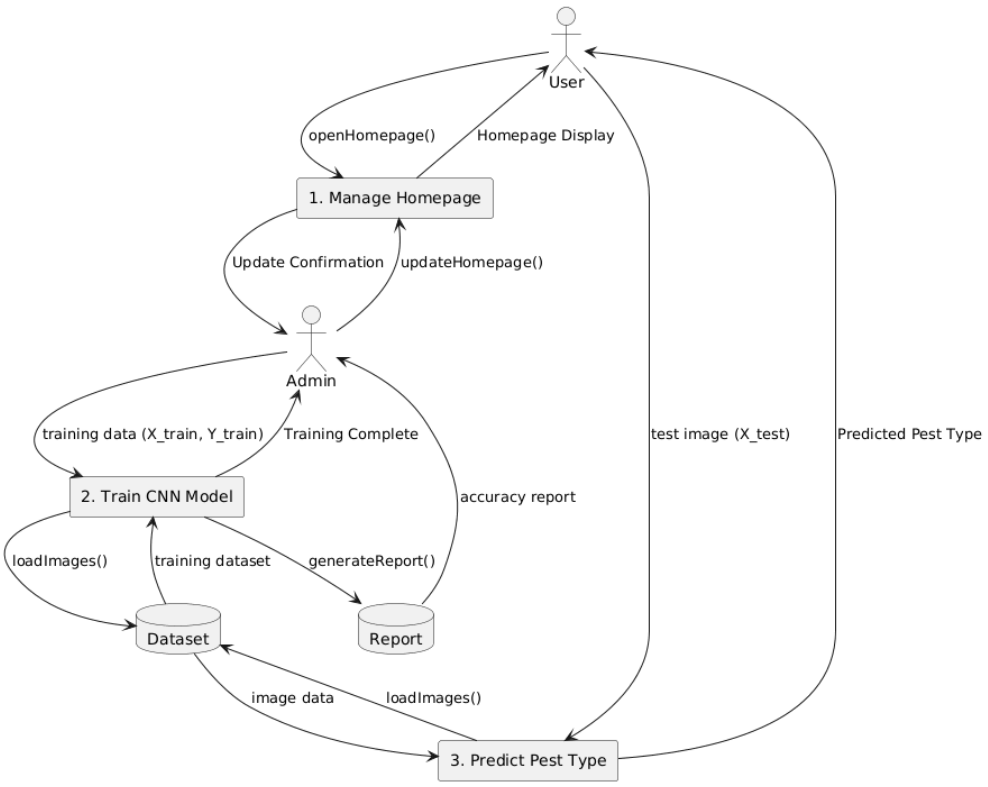

Fig 4.8 Data Flow Diagram for Classification of Pests.

IMPLEMENTATION

5. IMPLEMETATION

5.1 PYTHON

* One of the most popular languages is Python. Guido van Rossum released this language in 1991. Python is available on the Mac, Windows, and Raspberry Pi operating systems. The syntax of Python is simple and identical to that of English. When compared to Python, it was seen that the other language requires a few extra lines.

*It is an interpreter-based language because code may be run line by line after it has been written. This implies that rapid prototyping is possible across all platforms. Python is a big language with a free, binary-distributed interpreter standard library.

* It is inferior to maintenance that is conducted and is straightforward to learn. It is an object-oriented, interpreted programming language. It supports several different programming paradigms in addition to object-oriented programming, including functional and procedural programming.

* It supports several different programming paradigms in addition to object-oriented programming, including practical and procedural programming. Python is mighty while maintaining a relatively straightforward syntax. Classes, highly dynamic data types, modules, and exceptions are covered. Python can also be utilised by programmes that require programmable interfaces as an external language.

Here are some key features and characteristics of Python:

- Readability: Python emphasizes code readability with its clean and intuitive syntax. It uses indentation and whitespace to structure code blocks, making it easy to understand and maintain.
- Easy to Learn: Python's simplicity and readability make it an excellent choice for beginners. Its straightforward syntax and extensive documentation make it accessible for newcomers to programming.
- Interpreted Language: Python is an interpreted language, meaning that it doesn't need to be compiled before running. The Python interpreter reads and executes the code directly, making the development process faster and more interactive.

- Cross-platform Compatibility: Python is available for major operating systems like Windows, macOS, and Linux. This cross-platform compatibility allows developers to write code once and run it on different platforms without modifications.
- Large Standard Library: Python comes with a vast standard library that provides ready-to-use modules and functions for various tasks. It covers areas such as file I/O, networking, regular expressions, databases, and more, saving developers time and effort.
- Extensible and Modular: Python supports modular programming, enabling developers to organize code into reusable modules and packages. Additionally, Python allows integrating modules written in other languages, such as C or C++, providing flexibility and performance optimizations.
- Wide Range of Libraries and Frameworks: Python has a vibrant ecosystem with numerous third-party libraries and frameworks. These libraries, such as NumPy, pandas, TensorFlow, and Django, extend Python's capabilities for specific domains, making it a powerful tool for diverse applications.
- Object-Oriented: Python supports object-oriented programming (OOP) principles, allowing developers to create and work with classes and objects. OOP provides a structured approach to code organization, promoting code reuse and modularity.
- Dynamic Typing: Python is dynamically typed, meaning variable types are determined at runtime. Developers do not need to declare variable types explicitly, which enhances flexibility and simplifies code writing.

5.2 Installation

To install Python on your computer, follow these basic steps:

- Step 1: Visit the Python website Go to the official Python website at https://www.python.org/.
- Step 2: Select the operating system Choose the appropriate installer for your operating system. Python supports Windows, macOS, and various Linux distributions. Make sure to select the correct version that matches your operating system.
- Step 3: Check which version of Python is installed; if the 3.7.0 version is not there, uninstall it through the control panel and

- Step 4: Install Python 3.7.0 using Cmd.
- Step 5: Install the all libraries that required to run the project
- Step 6: Run

5.3 Python Features:

1) **Easy:** Because Python is a more accessible and straightforward language, Python programming is easier to learn.
2) **Interpreted language:** Python is an interpreted language, therefore it can be used to examine the code line by line and provide results.
3) **Open Source:** Python is a free online programming language since it is open-source.
4) **Portable:** Python is portable because the same code may be used on several computer standard
5) **libraries:** Python offers a sizable library that we may utilize to create applications quickly.
6) **GUI:** It stands for GUI (Graphical User Interface)
7) **Dynamical typed:** Python is a dynamically typed language, therefore the type of the value will be determined at runtime.

5.4 Python GUI (Tkinter)

- Python provides a wide range of options for GUI development (Graphical User Interfaces).
- Tkinter, the most widely used GUI technique, is used for all of them.
- The Tk GUI toolkit offered by Python is used with the conventional Python interface.
- Tkinter is the easiest and quickest way to write Python GUI programs.
- Using Tkinter, creating a GUI is simple.
- A part of Python's built-in library is Tkinter. The GUI programs were created.
- Python and Tkinter together give a straightforward and quick way. The Tk GUI toolkit's object-oriented user interface is called Tkinter.

Making a GUI application is easy using Tkinter. Following are the steps:

1) Install the Tkinter module in place.

2) The GUI applicatioMakeske the primary window

3) Include one or more of the widgets mentioned above in the GUI application.

4) Set up the main event loop such that it reacts to each user-initiated event.

Although Tkinter is the only GUI framework included in the Python standard library, Python includes a GUI framework. The default library for Python is called Tkinter. Tk is a scripting language often used in designing, testing, and developing GUIs. Tk is a free, open-source widget toolkit that may be used to build GUI applications in a wide range of computer languages.

5.5 Python IDLE

- Python IDLE offers a full-fledged file editor, which gives you the ability to write and execute Python programs from within this program. The built-in file editor also includes several features, like code completion and automatic indentation, that will speed up your coding workflow.
- Guido Van Rossum named Python after the British comedy group Monty Python while the name IDLE was chosen to pay tribute to Eric Idle, who was one of the Monty Python's founding members. IDLE comes bundled with the default implementation of the Python language since the 01.5. 2b1 release
- IDLE is used to execute statements similar to Python Shell. IDLE is used to create, modify, and execute Python code. IDLE provides a fully-featured text editor to write Python scripts and provides features like syntax highlighting, auto-completion, and smart indent.
- IDLE has two modes: interactive and script. We wrote our first program, “Hello, World!” in interactive mode. Interactive mode immediately returns the results of commands you enter into the shell. In script mode, you will write a script and then run it.
- The IDE Python IDLE is a good place to start as it helps you become familiar with the way Python works and understand its syntax. This IDE is good to start programming in Python due to its great debugger, but once you are fluent and start developing projects it is necessary to jump to another, more complete IDE.

- ❖ Python IDLE (Integrated Development and Learning Environment) is an interactive development environment included with the Python programming language. It provides a convenient way to write, execute, and debug Python code.

When you install Python, IDLE is typically installed along with it. To open IDLE, you can follow these steps:

- o Open the command prompt (Windows) or terminal (macOS/Linux).
- o Type "idle" and press Enter. Alternatively, you can specify the version with "idle3" or "idle2" for Python 3 or Python 2, respectively.
- o Once IDLE is launched, you will see the Python shell, which is an interactive environment where you can type and execute Python code directly.

Here are some features and functionalities provided by Python IDLE:

- Editor: IDLE includes a text editor where you can write your Python code. It offers syntax highlighting, automatic indentation, and code completion to enhance your coding experience.
- Interactive Shell: The Python shell in IDLE allows you to execute Python code interactively. You can type commands, statements, or function calls directly in the shell, and Python will execute them immediately.
- Debugging: IDLE provides basic debugging capabilities to help you find and fix errors in your code. You can set breakpoints, step through code, inspect variables, and track the program's execution.
- Python Help: IDLE provides access to the Python documentation and built-in help. You can access the help menu to find information about Python modules, functions, classes, and more.
- Script Execution: In addition to the interactive shell, IDLE allows you to run Python scripts stored in files. You can write your code in the editor and execute it as a script to see the output or interact with the program.
- Customization: IDLE can be customized to suit your preferences. You can modify settings related to syntax highlighting, indentation, fonts, and more.
- Python IDLE serves as a beginner-friendly development environment and learning tool. It is suitable for writing small scripts, testing code snippets, experimenting with

Python features, and learning the language's basics. However, for more advanced development projects, you may consider using other code editors or integrated development environments (IDEs) that provide additional features and better project management capabilities.

5.6 Libraries

In Python, libraries (also referred to as modules or packages) are collections of pre-written code that provide additional functionality and tools to extend the capabilities of the Python language. Libraries contain reusable code that developers can leverage to perform specific tasks without having to write everything from scratch.

Python libraries are designed to solve common problems, such as handling data, performing mathematical operations, interacting with databases, working with files, implementing networking protocols, creating graphical user interfaces (GUIs), and much more. They provide ready-to-use functions, classes, and methods that simplify complex operations and save development time.

Libraries in Python offer various advantages:

- Code Reusability:
- Efficiency:
- Collaboration
- Domain-Specific Functionality
- To use a Python library, you need to install it first.

There are some libraries following:

- **Pandas:**

Pandas are a Python computer language library for data analysis and manipulation. It offers a specific operation and data format for handling time series and numerical tables. It differs significantly from the release3-clause of the BSD license. It is a well-liked open-source of opinion that is utilized in machine learning and data analysis.

Pandas are a Python package providing fast, flexible, and expressive data structures designed to make working with “relational” or “labeled” data both easy and intuitive. It aims

to be the fundamental high-level building block for doing practical, real-world data analysis in Python. Pandas are a Python library used for working with data sets.

- It has functions for analysing, cleaning, exploring, and manipulating data.
- The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.
- Pandas allow us to analyse big data and make conclusions based on statistical theories.
- Pandas can clean messy data sets, and make them readable and relevant.

Relevant data is very important in data science. Pandas are a Python library for data analysis. Started by Wes McKinney in 2008 out of a need for a powerful and flexible quantitative analysis tool, pandas have grown into one of the most popular Python libraries. It has an extremely active community of contributors. The name is derived from the term "panel data", an econometrics term for data sets that include observations over multiple time periods for the same individuals. Its name is a play on the phrase "Python data analysis" itself.

- **NumPy:**

The NumPy Python library for multi-dimensional, big-scale matrices adds a huge number of high-level mathematical functions. It is possible to modify NumPy by utilizing a Python library. Along with line, algebra, and the Fourier transform operations, it also contains several matrices-related functions.

NumPy can be used to perform a wide variety of mathematical operations on arrays. It adds powerful data structures to Python that guarantee efficient calculations with arrays and matrices and it supplies an enormous library of high-level mathematical functions that operate on these arrays and matrices.

- NumPy is a Python library used for working with arrays.
- It also has functions for working in domain of linear algebra, Fourier transform, and matrices.
- NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

- NumPy stands for Numerical Python.
- In Python we have lists that serve the purpose of arrays, but they are slow to process.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.
- Arrays are very frequently used in data science, where speed and resources are very important.

➢ **Matplotlib:**

It is a multi-platform, array-based data visualization framework built to interact with the whole SciPy stack. MATLAB is proposed as an open-source alternative. Matplotlib is a Python extension and a cross-platform toolkit for graphical plotting and visualization.

Matplotlib is a popular Python library for creating static, animated, and interactive visualizations. It provides a flexible and comprehensive set of tools for generating plots, charts, histograms, scatter plots, and more. Matplotlib is widely used in various fields, including data analysis, scientific research, and data visualization.

Here are some key features and functionalities of the Matplotlib library:

- Plotting Functions
- Customization Options
- Multiple Interfaces
- Integration with NumPy and pandas
- Subplots and Figures:
- Saving and Exporting

➢ **Scikit-learn:**

The most stable and practical machine learning library for Python is scikit-learn. Regression, dimensionality reduction, classification, and clustering are just a few of the

helpful tools it provides through the Python interface for statistical modelling and machine learning. It is an essential part of the Python machine learning toolbox used by JP Morgan. It is frequently used in various machine learning applications, including classification and predictive analysis.

Scikit-learn (also referred to as sklearn) is a widely used open-source machine learning library for Python. It provides a comprehensive set of tools and algorithms for various machine learning tasks, including classification, regression, clustering, dimensionality reduction, model selection, and pre-processing.

Here are some key features and functionalities of the Scikit-learn library:

- Easy-to-Use Interface:
- Broad Range of Algorithms:
- Data Pre-processing and Feature Engineering:
- Model Evaluation and Validation:
- Integration with NumPy and pandas:
- Robust Documentation and Community Support:

➢ **Keras:**

* Google's Keras is a cutting-edge deep learning API for creating neural networks. It is created in Python and is designed to simplify the development of neural networks. Additionally, it enables the use of various neural networks for computation. Deep learning models are developed and tested using the free and open-source Python software known as Keras.

Keras is a high-level deep learning library for Python. It is designed to provide a user-friendly and intuitive interface for building and training deep learning models. Keras acts as a front-end API, allowing developers to define and configure neural networks while leveraging the computational backend engines, such as Tensor Flow or Theano.

Here are some key features and functionalities of the Keras library:

- User-Friendly API
- Multi-backend Support

- Wide Range of Neural Network Architectures
- Pre-trained Models and Transfer Learning:
- Easy Model Training and Evaluation:
- GPU Support:

➢ **h5py:**

* The h5py Python module offers an interface for the binary HDF5 data format. Thanks to p5py, the top can quickly halt the vast amount of numerical data and alter it using the NumPy library. It employs common syntax for Python, NumPy, and dictionary arrays.

h5py is a Python library that provides a simple and efficient interface for working with datasets and files in the Hierarchical Data Format 5 (HDF5) format. HDF5 is a versatile data format commonly used for storing and managing large volumes of numerical data.

Here are some key features and functionalities of the h5py library:

- HDF5 File Access
- Dataset Handling:
- Group Organization:
- Attributes:
- Compatibility with NumPy
- Performance

➢ **Tensor flow**

TensorFlow is a Python library for fast numerical computing created and released by Google. It is a foundation library that can be used to create Deep Learning models directly or by using wrapper libraries that simplify the process built on top of TensorFlow. TensorFlow is an end-to-end open source platform for machine learning. TensorFlow is a rich system for managing all aspects of a machine learning system; however, this class focuses on using a particular TensorFlow API to develop and train machine learning models.

TensorFlow is a popular open-source library for machine learning and deep learning. It provides a comprehensive set of tools, APIs, and computational resources for building and training various types of machine learning models, especially neural networks.

Here are some key features and functionalities of TensorFlow:

- Neural Network Framework:
- Computational Graphs
- Automatic Differentiation
- GPU and TPU Support
- Distributed Computing
- Deployment Capabilities

➢ **Tkinter**

Tkinter is an acronym for "Tk interface". Tk was developed as a GUI extension for the Tcl scripting language by John Ousterhout. The first release was in 1991. Tkinter is the de facto way in Python to create Graphical User interfaces (GUIs) and is included in all standard Python Distributions. In fact, it's the only framework built into the Python standard library.

Tkinter is a standard Python library used for creating graphical user interfaces (GUIs). It provides a set of modules and classes that allow you to develop interactive and visually appealing desktop applications.

Here are some key features and functionalities of Tkinter:

- Cross-Platform Compatibility
- Simple and Easy-to-Use
- Widgets and Layout Management
- Event-Driven Programming
- Customization and Styling
- Integration with Other Libraries

- **NLTK**

NLTK is a toolkit build for working with NLP in Python. It provides us various text processing libraries with a lot of test datasets. A variety of tasks can be performed using NLTK such as tokenizing, parse tree visualization, etc NLTK (Natural Language Toolkit) is the go-to API for NLP (Natural Language Processing) with Python. It is a really powerful tool to pre-process text data for further analysis like with ML models for instance. It helps convert text into numbers, which the model can then easily work with.

NLTK (Natural Language Toolkit) is a Python library widely used for working with human language data and implementing natural language processing (NLP) tasks. It provides a set of tools, corpora, and resources for tasks such as tokenization, stemming, tagging, parsing, sentiment analysis, and more.

Here are some key features and functionalities of NLTK:

- Text Processing
- Part-of-Speech Tagging
- Named Entity Recognition
- Chunking and Parsing
- Sentiment Analysis:
- WordNet Integration:

- **Scipy**

SciPy is a collection of mathematical algorithms and convenience functions built on the NumPy extension of Python. It adds significant power to the interactive Python session by providing the user with high-level commands and classes for manipulating and visualizing data.

SciPy is a powerful scientific computing library for Python that provides a wide range of mathematical algorithms and functions. It builds upon NumPy, another fundamental library for numerical computing, and extends its capabilities by adding additional tools for scientific and technical computing tasks.

Here are some key features and functionalities of SciPy:

- Numerical Integration:
- Optimization and Root Finding
- Linear Algebra
- Signal and Image Processing
- Statistics

5.7 ALGORITHMS:

Convolutional Neural Network (CNN)

A **Convolutional Neural Network (CNN)** is a type of Deep Learning neural network architecture commonly used in Computer Vision. Computer vision is a field of Artificial Intelligence that enables a computer to understand and interpret the image or visual data.

When it comes to Machine Learning, Artificial Neural Networks perform really well. Neural Networks are used in various datasets like images, audio, and text. Different types of Neural Networks are used for different purposes, for example for predicting the sequence of words we use **Recurrent Neural Networks** more precisely an LSTM, similarly for image classification we use Convolution Neural networks. In this blog, we are going to build a basic building block for CNN.

In a regular Neural Network there are three types of layers:

1. **Input Layers:** It's the layer in which we give input to our model. The number of neurons in this layer is equal to the total number of features in our data (number of pixels in the case of an image).
2. **Hidden Layer:** The input from the Input layer is then fed into the hidden layer. There can be many hidden layers depending on our model and data size. Each hidden layer can have different numbers of neurons which are generally greater than the number of features. The output from each layer is computed by matrix multiplication of the output of the previous layer with learnable weights of that layer and then by the addition of learnable biases followed by activation function which makes the network nonlinear.
3. **Output Layer:** The output from the hidden layer is then fed into a logistic function like sigmoid or softmax which converts the output of each class into the probability score of each class.

The data is fed into the model and output from each layer is obtained from the above step is called **feedforward**, we then calculate the error using an error function, some common error functions are cross-entropy, square loss error, etc. The error function measures how well the network is performing. After that, we backpropagate into the model by calculating the derivatives. This step is called **Backprzopagation** which basically is used to minimize the loss.

Convolution Neural Network

Convolutional Neural Network (CNN) is the extended version of artificial neural networks (ANN) which is predominantly used to extract the feature from the grid-like matrix dataset. For example visual datasets like images or videos where data patterns play an extensive role.

CNN architecture

Convolutional Neural Network consists of multiple layers like the input layer, Convolutional layer, Pooling layer, and fully connected layers.

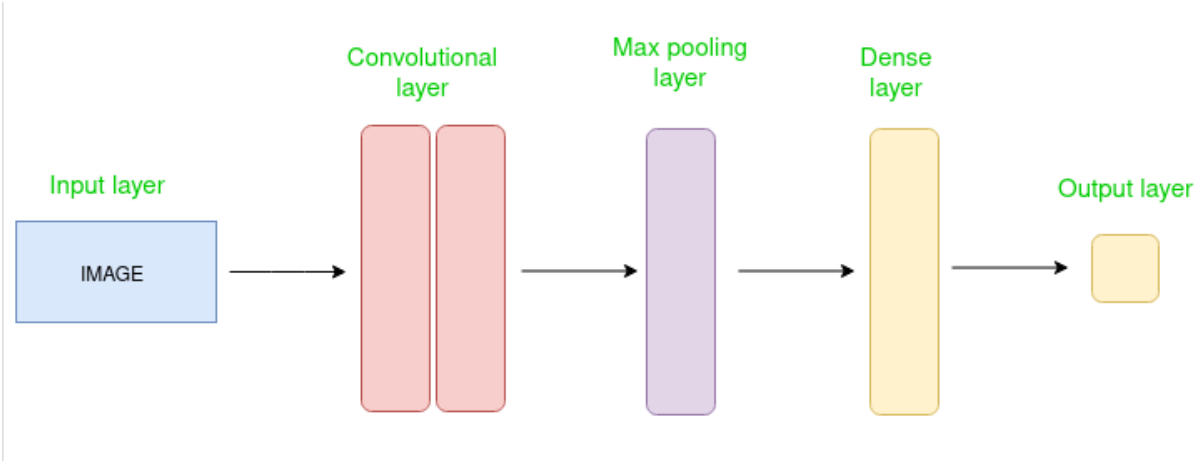

Fig 5.7.1: Convolutional Neural Network (CNN) Architecture

The Convolutional layer applies filters to the input image to extract features, the Pooling layer downsamples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.

How Convolutional Layers works

Convolution Neural Networks or covnets are neural networks that share their parameters. Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (i.e the channel as images generally have red, green, and blue channels).

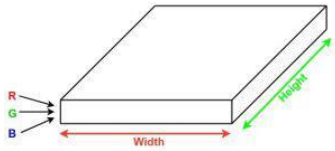

Fig 5.7.2: Image Representation in Terms of Dimensions and Color Channels

Now imagine taking a small patch of this image and running a small neural network, called a filter or kernel on it, with say, K outputs and representing them vertically. Now slide that neural network across the whole image, as a result, we will get another image with different widths, heights, and depths. Instead of just R, G, and B channels now we have more channels but lesser width and height. This operation is called **Convolution**. If the patch size is the same as that of the image it will be a regular neural network. Because of this small patch, we have fewer weights.

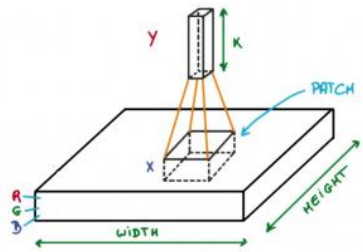

Fig 5.7.3: Patch Antenna Geometry and Coordinate Representation

Image source: Deep Learning Udacity

Now let's talk about a bit of mathematics that is involved in the whole convolution process.

- Convolution layers consist of a set of learnable filters (or kernels) having small widths and heights and the same depth as that of input volume (3 if the input layer is image input).
- For example, if we have to run convolution on an image with dimensions 34x34x3. The possible size of filters can be axax3, where 'a' can be anything like 3, 5, or 7 but smaller as compared to the image dimension.
- During the forward pass, we slide each filter across the whole input volume step by step where each step is called **stride** (which can have a value of 2, 3, or even 4 for high-dimensional images) and compute the dot product between the kernel weights and patch from input volume.

- As we slide our filters we'll get a 2-D output for each filter and we'll stack them together as a result, we'll get output volume having a depth equal to the number of filters. The network will learn all the filters.

Layers used to build ConvNets

A complete Convolution Neural Networks architecture is also known as covnets. A covnets is a sequence of layers, and every layer transforms one volume to another through a differentiable function.

Types of layers:

Let's take an example by running a covnets on of image of dimension 32 x 32 x 3.

- **Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.
- **Convolutional Layers:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2×2, 3×3, or 5×5 shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension 32 x 32 x 12.
- **Activation Layer:** By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are **RELU**: max(0, x), **Tanh**, **Leaky RELU**, etc. The volume remains unchanged hence output volume will have dimensions 32 x 32 x 12.
- **Pooling layer:** This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are **max pooling** and **average pooling**. If we use a max pool with 2 x 2 filters and stride 2, the resultant volume will be of dimension 16x16x12.

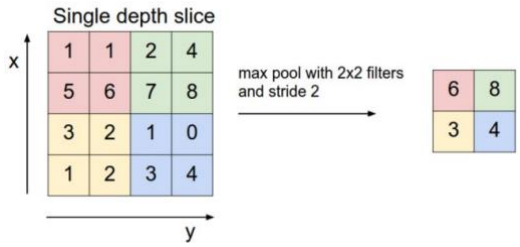

Fig 5.7.4: Max Pooling Operation in Convolutional Neural Networks

Image source: cs231n.stanford.edu

- **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
- **Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.
- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.

5.8 Sample Code:

Views.py:

```
from django.shortcuts import render

from django.template import RequestContext

from django.contrib import messages

from django.http import HttpResponse

import os

import pickle

from django.core.files.storage import FileSystemStorage

import cv2

import numpy as np
```

```
import matplotlib.pyplot as plt

from keras.models import *

import io

import base64


from keras.utils import to_categorical

from keras.layers import  MaxPooling2D

from keras.layers import Dense, Dropout, Activation, Flatten

from keras.layers import Convolution2D

from keras.models import Sequential, load_model, Model

from sklearn.metrics import precision_score

from sklearn.metrics import recall_score

from sklearn.metrics import f1_score

from sklearn.metrics import accuracy_score

from sklearn.model_selection import train_test_split

from keras.callbacks import ModelCheckpoint

import keras


global uname


path = "Dataset"

labels = []

X = []
```

```
Y = []

for root, dirs, directory in os.walk(path):
    for j in range(len(directory)):
        name = os.path.basename(root)
        if name not in labels:
            labels.append(name.strip())

X = np.load('model/X.txt.npy')
Y = np.load('model/Y.txt.npy')

X = X.astype('float32')
X = X/255

indices = np.arange(X.shape[0])
np.random.shuffle(indices)
X = X[indices]
Y = Y[indices]
Y = to_categorical(Y)

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2) #split dataset into train
and test
cnn_model = Sequential()
```

```
cnn_model.add(Convolution2D(64, (3, 3), input_shape = (X_train.shape[1], X_train.shape[2], X_train.shape[3]), activation = 'relu'))

cnn_model.add(MaxPooling2D(pool_size = (2, 2)))

cnn_model.add(Convolution2D(28, (3, 3), activation = 'relu'))

cnn_model.add(MaxPooling2D(pool_size = (2, 2)))

cnn_model.add(Flatten())

cnn_model.add(Dense(units = 256, activation = 'relu'))

cnn_model.add(Dense(units = y_train.shape[1], activation = 'softmax'))

cnn_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics = ['accuracy'])

if os.path.exists("model/cnn_weights.hdf5") == False:

    model_check_point = ModelCheckpoint(filepath='model/cnn_weights.hdf5', verbose = 1, save_best_only = True)

    hist = cnn_model.fit(X_train, y_train, batch_size = 32, epochs = 35, validation_data=(X_test, y_test), callbacks=[model_check_point], verbose=1)

    f = open('model/cnn_history.pckl', 'wb')

    pickle.dump(hist.history, f)

    f.close()

else:

    cnn_model.load_weights("model/cnn_weights.hdf5")


predict = cnn_model.predict(X_test)

predict = np.argmax(predict, axis=1)

y_test1 = np.argmax(y_test, axis=1)

a = accuracy_score(y_test1,predict)*100
```

```
p = precision_score(y_test1, predict,average='macro') * 100
r = recall_score(y_test1, predict,average='macro') * 100
f = f1_score(y_test1, predict,average='macro') * 100


def predict(filename, cnn_model):
    global labels
    img = cv2.imread(filename)
    img = cv2.resize(img, (32,32))#resize image
    im2arr = np.array(img)
    im2arr = im2arr.reshape(1,32,32,3)
    img = np.asarray(im2arr)
    img = img.astype('float32')
    img = img/255 #normalizing test image
    predict = cnn_model.predict(img)#now using  efficient_model to predict pest
    predict = np.argmax(predict)
    img = cv2.imread(filename)
    img = cv2.resize(img, (600,400))
    cv2.putText(img, 'Pest Classified As : '+labels[predict], (10, 25),
cv2.FONT_HERSHEY_SIMPLEX,0.7, (255, 0, 0), 2)
    return img, 'Pest Classified As : '+labels[predict]


def predictModelAction(request):
    if request.method == 'POST':
        cnn_model = load_model("model/cnn_weights.hdf5")
```

```
        myfile = request.FILES['t1'].read()
        fname = request.FILES['t1'].name
        if os.path.exists("ClassificationApp/static/test.jpg"):
            os.remove("ClassificationApp/static/test.jpg")
        with open("ClassificationApp/static/test.jpg", "wb") as file:
            file.write(myfile)
        file.close()
        img, msg = predict("ClassificationApp/static/test.jpg", cnn_model)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.imshow(img)
        buf = io.BytesIO()
        plt.savefig(buf, format='png', bbox_inches='tight')
        plt.close()
        img_b64 = base64.b64encode(buf.getvalue()).decode()
        context= {'data': msg, 'img': img_b64}
        return render(request, 'index.html', context)


def index(request):
    if request.method == 'GET':
        output = "Classification of Pests using Computer Vision CNN Algorithm"
        context= {'data': output}
        return render(request, 'index.html', context)
```

```
def predictModel(request):
    if request.method == 'GET':
        return render(request, 'predictModel.html', {})


def trainModel(request):
    if request.method == 'GET':
        global a, p, r, f
        output ='<table border=1 align=center width=100%><tr><th><font size="" color="black">Algorithm Name</th><th><font size="" color="black">Accuracy</th><th><font size="" color="black">Precision</th>'
        output+='<th><font size="" color="black">Recall</th><th><font size="" color="black">FSCORE</th></tr>'
        global accuracy, precision, recall, fscore
        algorithms = ['CNN Algorithm']
        output+='<tr><td><font size="" color="black">'+algorithms[0]+'</td><td><font size="" color="black">'+str(a)+'</td><td><font size="" color="black">'+str(p)+'</td><td><font size="" color="black">'+str(r)+'</td><td><font size="" color="black">'+str(f)+'</td></tr>'
        output+= "</table></br>"
        f = open('model/cnn_history.pckl', 'rb')
        data = pickle.load(f)
        f.close()
        accuracy = data['accuracy']
        loss = data['loss']
        plt.figure(figsize= (6,4))
```

```
    plt.grid(True)
    plt.xlabel('Training Epoch')
    plt.ylabel('Accuracy/Loss')
    plt.plot(loss, 'ro-', color = 'red')
    plt.plot(accuracy, 'ro-', color = 'green')
    plt.legend(['Loss', 'Accuracy'], loc='upper left')
    plt.title('CNN Training Accuracy & Loss Graph')
    plt.tight_layout()
    buf = io.BytesIO()
    plt.savefig(buf, format='png', bbox_inches='tight')
    plt.close()
    img_b64 = base64.b64encode(buf.getvalue()).decode()
    context= {'data': output, 'img': img_b64}
    return render(request, 'index.html', context)
```

Manage.py:

```
#!/usr/bin/env python
import os
import sys


if __name__ == '__main__':
  os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'Classification.settings')
  try:
    from django.core.management import execute_from_command_line
```

```
except ImportError as exc:
    raise ImportError(
        "Couldn't import Django. Are you sure it's installed and "
        "available on your PYTHONPATH environment variable? Did you "
        "forget to activate a virtual environment?"
    ) from exc
execute_from_command_line(sys.argv)
```

TESTING

6. TESTING

Implementation and Testing

Implementation is one of the most important tasks in project is the phase in which one has to be cautions because all the efforts undertaken during the project will be very interactive. Implementation is the most crucial stage in achieving successful system and giving the users confidence that the new system is workable and effective. Each program is tested individually at the time of development using the sample data and has verified that these programs link together in the way specified in the program specification. The computer system and its environment are tested to the satisfaction of the user.

Implementation

The implementation phase is less creative than system design. It is primarily concerned with user training, and file conversion. The system may be requiring extensive user training. The initial parameters of the system should be modifies as a result of a programming. A simple operating procedure is provided so that the user can understand the different functions clearly and quickly. The different reports can be obtained either on the inkjet or dot matrix printer, which is available at the disposal of the user. The proposed system is very easy to implement. In general implementation is used to mean the process of converting a new or revised system design into an operational one.

Testing

Testing is the process where the test data is prepared and is used for testing the modules individually and later the validation given for the fields. Then the system testing takes place which makes sure that all components of the system property functions as a unit. The test data should be chosen such that it passed through all possible condition. Actually, testing is the state of implementation which aimed at ensuring that the system works accurately and efficiently before the actual operation commence. The following is the description of the testing strategies, which were carried out during the testing period.

System Testing

Testing has become an integral part of any system or project especially in the field of information technology. The importance of testing is a method of justifying, if one is ready to move further, be it to be check if one is capable to with stand the rigors of a particular situation cannot be underplayed and that is why testing before development is so critical. When the software is developed before it is given to user to use the software must be tested whether it is solving the purpose for which it is developed. This testing involves various types through which one can ensure the software is reliable. The program was tested logically and pattern of execution of the program for a set of data are repeated.

Module Testing

To locate errors, each module is tested individually. This enables us to detect error and correct it without affecting any other modules. Whenever the program is not satisfying the required function, it must be corrected to get the required result. Thus, all the modules are individually tested from bottom up starting with the smallest and lowest modules and proceeding to the next level. Each module in the system is tested separately. For example, the job classification module is tested separately. This module is tested with different job and its approximate execution time and the result of the test is compared with the results that are prepared manually. The comparison shows that the results proposed system works efficiently than the existing system. Each module in the system is tested separately.

Integration Testing

After the module testing, the integration testing is applied. When linking the modules there may be chance for errors to occur, these errors are corrected by using this testing. In this system all modules are connected and tested. The testing results are very correct. Thus, the mapping of jobs with resources is done correctly by the system.

Acceptance Testing

When that user fined no major problems with its accuracy, the system passers through a final acceptance test. This test confirms that the system needs the original goals, objectives and requirements established during analysis without actual execution which

elimination wastage of time and money acceptance tests on the shoulders of users and management, it is finally acceptable and ready for the operation

Test Case Id	Test Case Name	Test Case Desc.	Test Steps			Test Case Status	Test Priority
			Step	Expected	Actual		
01	Home	Test whether the Homepage is uploaded or not into the system	Homepage may not upload	we cannot do further operations	Homepage uploaded we will do further operations	High	High
02	Train Pest Classification CNN Model	Verify the CNN Model Trained or not	Without Trained CNN Model	We cannot do further operations	We Can Trained CNN Model successfully We will do further operations	High	High
03	Predict Pest Type using Test Image	Predict Pest Type using Test Image will run or not	Without Predicting Pest Type using Test Image	We cannot do further operations	we can Predict Pest Type using Test Image We will do further operations	High	High

Table 6.1: Test Case's

RESULT

7.RESULT

To run project double, click on ‘run.bat’ file to start python server and get below page

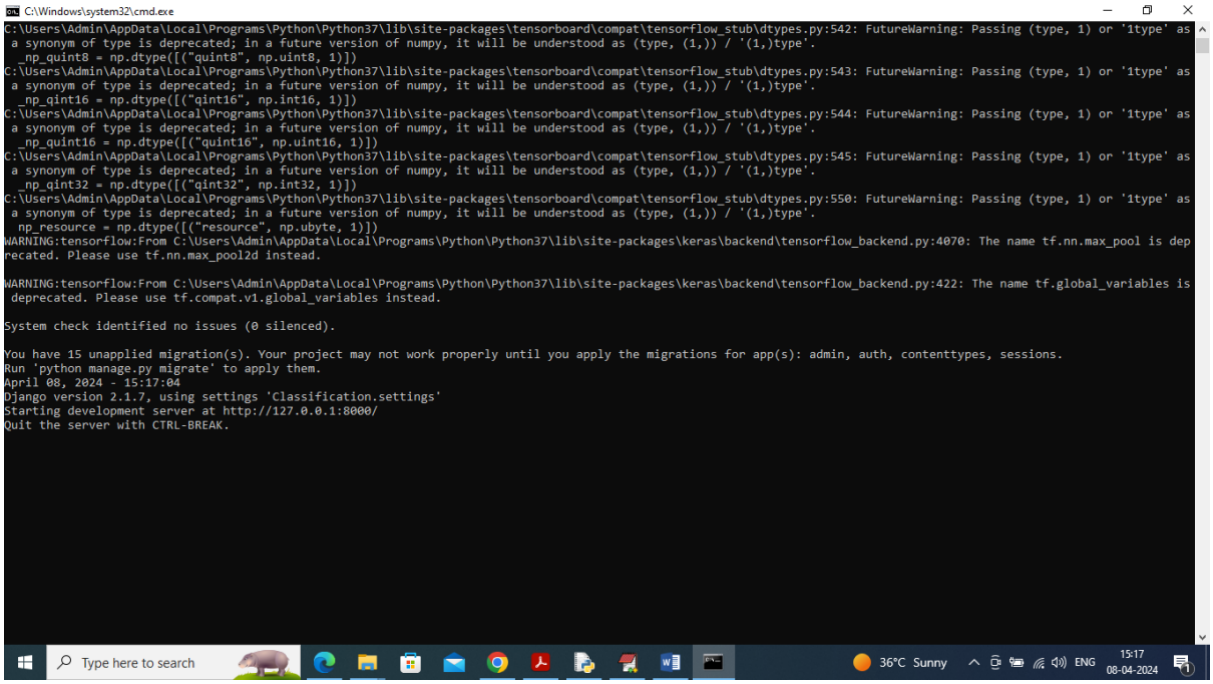

Fig 7.1: In above screen python server started and now open browse and enter URL as http://127.0.0.1:8000/index.html and press enter key to get below page

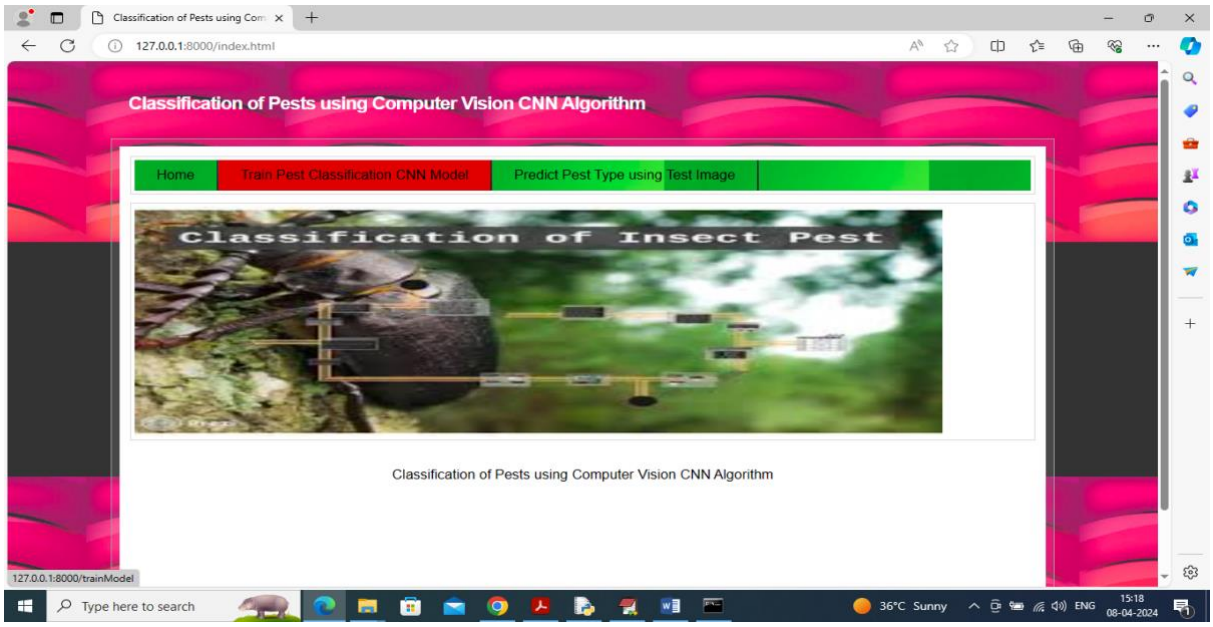

Fig 7.2: In above screen user can click on 'Train Pest Classification CNN Model' link to train CNN and get below page

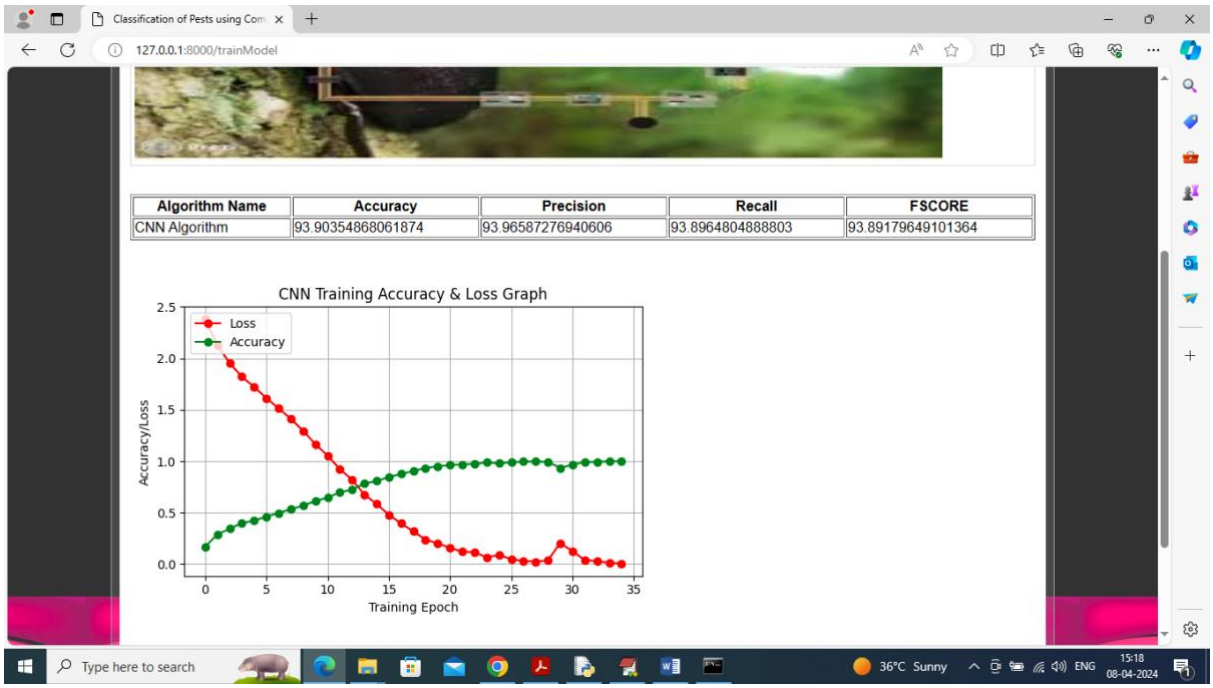

Fig 7.3: In above screen CNN model training completed and can see its accuracy as 93% and can see other metrics as Precision, Recall and FSCORE. In above graph we are plotting CNN training accuracy and loss values. In graph x-axis represents Training Epoch and y-axis represents accuracy/loss values and then red line represents loss and green line represents accuracy. In above graph can see with each increasing epoch accuracy got increase and reached closer to 1 and loss got decreased and reached closer to 0. Now click on 'Predict Pest Type using Test Image' link to get below page

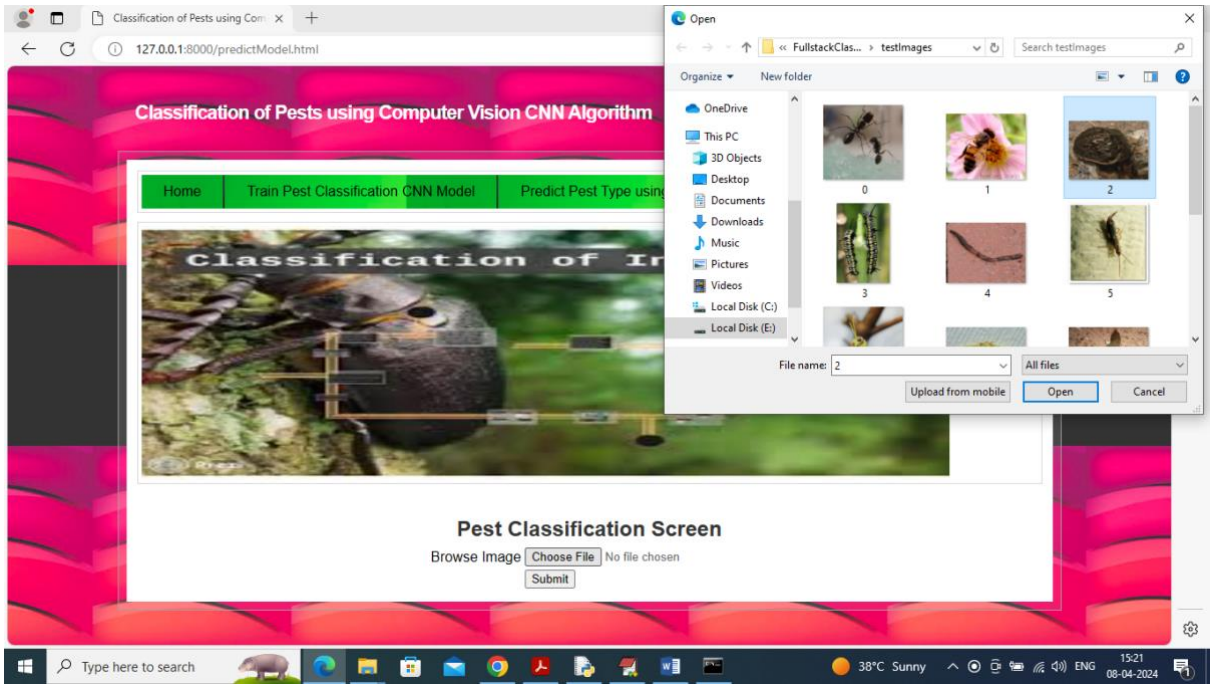

Fig 7.4: In above screen click on ''Choose File' and then upload test image and then click on 'Open' and 'Submit' button to upload image to server and get below classification output

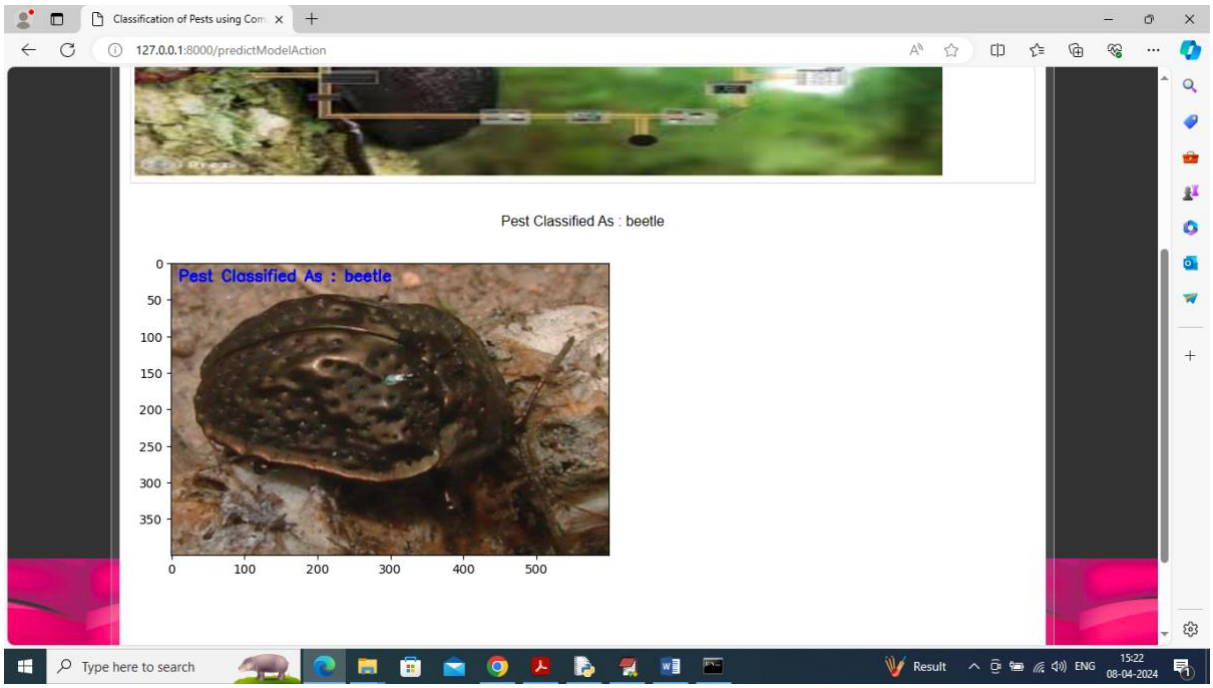

Fig 7.5: In above screen in blue colour text on image can see pest classify as 'beetle' and similarly you can upload and test other images

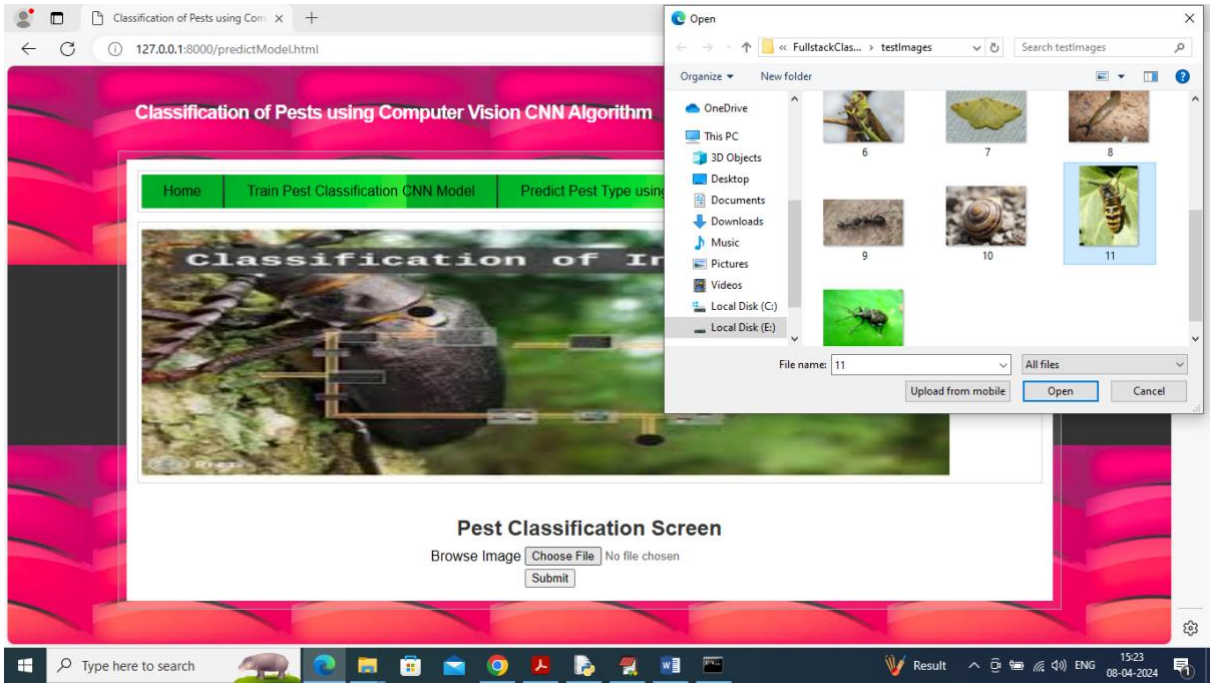

Fig 7.6: In above screen uploading another image and then press button to get below output

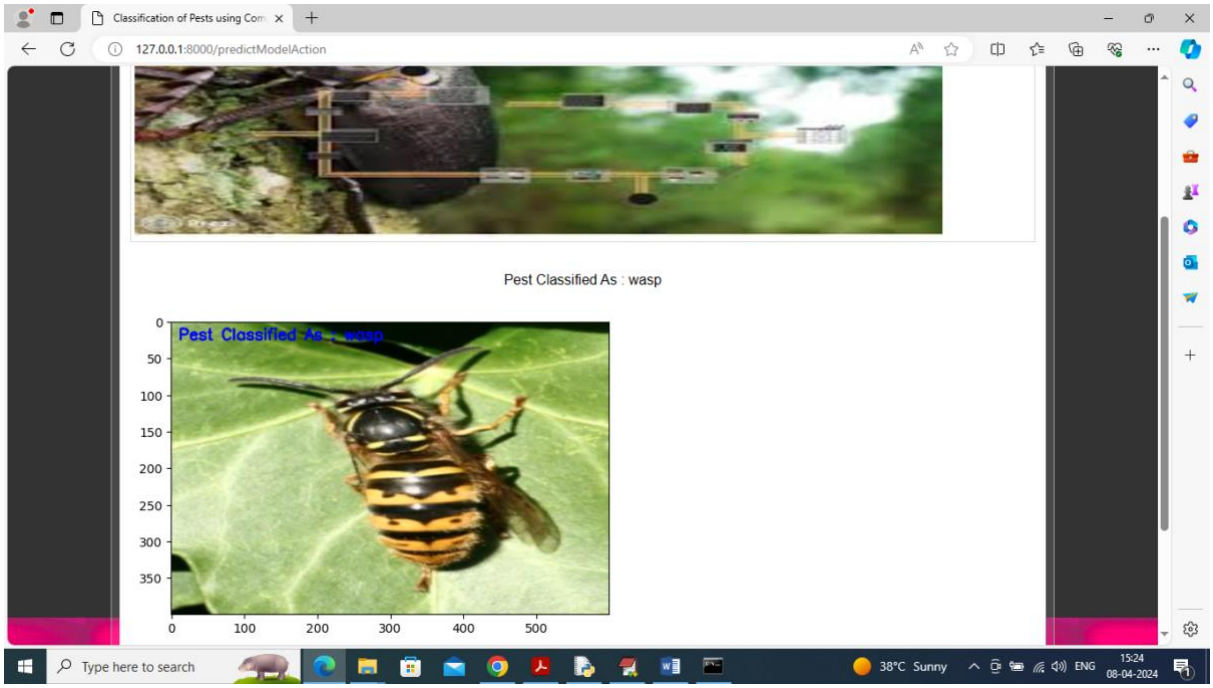

Fig 7.7: In above screen pest classified as 'wasp'.

CONCLUSION

8. CONCLUSION

Using the Google Net architecture, this research tries to apply the CNN method to develop an application that can detect pests in plants. In its implementation, this research showed that the convolution function, which extracted features pixel by pixel in the image, allowed CNN to detect pests on plants. Adding a thick layer improved the accuracy of the model; the number of thick layers made the model extract more features from the dataset, but adding too many thick layers caused the overfitting of the model. The layer dropout value of 0.5 was the most accurate but did not show significant changes between models. The model with an accuracy of 93.78% was the best in this research. By entering the Google Net model into an application, an application can be created to detect pests in plants on Android devices. This application was developed on a smartphone basis so that Indonesian farmers could easily identify pests on their crops. This application is also presented in Indonesian, which is the national language of Indonesia, so it is easy to use. It is the main contribution of this research. It is hoped that this application will be a breakthrough for Indonesian farmers. By identifying pests more easily, farmers can treat pests more appropriately, which will ultimately result in better crop yields. This research aims to help researchers and practitioners face the challenges that come with real-world implementation by covering topics such as more heterogeneous data, a focus on performance in limited-power devices, integration in end-to-end systems, security, end-user evaluation, the development of custom algorithms, and ongoing evaluation. This study has the potential to be extended further into applications that are more practical by increasing the scale of the experiment. This is also supported by an interview conducted with a respondent who is a staff member from the Plant Quarantine Agency of the Indonesian government. The suggestion is that the impact can be enhanced if the pests listed for detection also encompass the sorts of pests that currently exist in Indonesia but are still prevalent in specific regions. Due to the current process, this task is currently being performed manually by the agency. Therefore, it is possible to ensure that this research offers maximum advantages in a variety of practical applications while ensuring security, privacy, and a good influence on society if a commitment is made to following the relevant rules and standards and taking an ethical approach.

REFERENCES

9.REFERENCES

1. Rozaki, Z.; Wijaya, O.; Rahmawati, N.; Rahayu, L. Farmers' disaster mitigation strategies in Indonesia. Rev. Agric. Sci. 2021, 9, 178–194. [CrossRef]

2. Gandharum, L.; Mulyani, M.E.; Hartono, D.M.; Karsidi, A.; Ahmad, M. Remote sensing versus the area sampling frame method in paddy rice acreage estimation in Indramayu regency, West Java province, Indonesia. Int. J. Remote Sens. 2021, 42, 1738–1767. [CrossRef]

3. Yatoo, A.M.; Ali, M.N.; Baba, Z.A.; Hassan, B. Sustainable management of diseases and pests in crops by vermicompost and vermicompost tea. A review. Agron. Sustain. Dev. 2021, 41, 7. [CrossRef]

4. Nuryitmawan, T.R. The Impact of Credit on Multidimensional Poverty in Rural Areas: A Case Study of the Indonesian Agricultural Sector. Agriecobis J. Agric. Socioecon. Bus. 2021, 4, 32–45. [CrossRef]

5. Safitri, K.I.; Abdoellah, O.S.; Gunawan, B.; Suparman, Y.; Mubarak, A.Z.; Pardede, M. The Adaptation of Export-Scale Urban Farmers Amid the COVID-19 Pandemic in Bandung Metropolitan. Qual. Rep. 2022, 27, 1169–1196. [CrossRef]

6. Agustina, K.K.; Wirawan, I.M.A.; Sudarmaja, I.M.; Subrata, M.; Dharmawan, N.S. The first report on the prevalence of soiltransmitted helminth infections and associated risk factors among traditional pig farmers in Bali Province, Indonesia. Vet. World 2022, 15, 1154. [CrossRef]

7. Thao, L.Q.; Cuong, D.D.; Anh, N.T.; Minh, N.; Tam, N.D. Pest Early Detection in Greenhouse Using Machine Learning. Rev. D'intelligence Artif. 2022, 36, 209–214. [CrossRef]

8. Wang, X.; Liu, J.; Zhu, X. Early real-time detection algorithm of tomato diseases and pests in the natural environment. Plant Methods 2021, 17, 43. [CrossRef]

9. Gupta, N.; Slawson, D.D.; Moffat, A.J. Using citizen science for early detection of tree pests and diseases: Perceptions of professional and public participants. Biol. Invasions 2022, 24, 123–138. [CrossRef]

10. Pocock, M.J.; Marzano, M.; Bullas-Appleton, E.; Dyke, A.; De Groot, M.; Shuttleworth, C.M.; White, R. Ethical dilemmas when using citizen science for early detection of invasive tree pests and diseases. Manag. Biol. Invasions 2020, 11, 720–732. [CrossRef]

11. White, R.; Marzano, M.; Fesenko, E.; Inman, A.; Jones, G.; Agstner, B.; Mumford, R. Technology development for the early detection of plant pests: A framework for assessing Technology Readiness Levels (TRLs) in environmental science. J. Plant Dis. Prot. 2022, 129, 1249–1261. [CrossRef]

12. Rempelos, L.; Baranski, M.; Wang, J.; Adams, T.N.; Adebusuyi, K.; Beckman, J.J.; Brockbank, C.J.; Douglas, B.S.; Feng, T.; Greenway, J.D.; et al. Integrated soil and crop management in organic agriculture: A logical framework to ensure food quality and human health? Agronomy 2021, 11, 2494. [CrossRef]

13. Braga, M.P.; Janz, N. Host repertoires and changing insect–plant interactions. Ecol. Entomol. 2021, 46, 1241–1253. [CrossRef]

14. Yulita, I.N.; Amri, N.A.; Hidayat, A. Mobile Application for Tomato Plant Leaf Disease Detection Using a Dense Convolutional Network Architecture. Computation 2023, 11, 20. [CrossRef]

15. van der Merwe, D.; Burchfield, D.R.; Witt, T.D.; Price, K.P.; Sharda, A. Drones in agriculture. Adv. Agron. 2020, 162, 1. [CrossRef]

16. Dutta, G.; Goswami, P. Application of drone in agriculture: A review. Int. J. Chem. Stud. 2020, 8, 203–218. [CrossRef]

17. Ahirwar, S.; Swarnkar, R.; Bhukya, S.; Namwade, G. Application of Drone in Agriculture. Int. J. Curr. Microbiol. Appl. Sci. 2019, 8, 2500–2505. [CrossRef]

18. Rejeb, A.; Abdollahi, A.; Rejeb, K.; Treiblmaier, H. Drones in agriculture: A review and bibliometric analysis. Comput. Electron. Agric. 2022, 198, 107017. [CrossRef]

19. Tannous, M.; Stefanini, C.; Romano, D. A Deep-Learning-Based Detection Approach for the Identification of Insect Species of Economic Importance. Insects 2023, 14, 148. [CrossRef]

20. Gondal, M.D.; Khan, Y.N. Early pest detection from crop using image processing and computational intelligence. Int. J. Sci. Res. Eng. Manag. 2022, 6, 59–68. [CrossRef]

21. Hadipour-Rokni, R.; Asli-Ardeh, E.A.; Jahanbakhshi, A.; Esmaili paeen-Afrakoti, I.; Sabzi, S. Intelligent detection of citrus fruit pests using machine vision system and convolutional neural network through transfer learning technique. Comput. Biol. Med. 2023, 155, 106611. [CrossRef]

22. Kasinathan, T.; Singaraju, D.; Uyyala, S.R. Insect classification and detection in field crops using modern machine learning techniques. Inf. Process. Agric. 2021, 8, 446–457. [CrossRef]

23. Barbedo, J.G.A. Detecting and Classifying Pests in Crops Using Proximal Images and Machine Learning: A Review. AI 2020, 1, 21. [CrossRef]

24. Sangeetha, T.; Mohanapriya, M. A Novel Exploration of Plant Disease and Pest Detection Using Machine Learning and Deep Learning Algorithms. Publ. Issue 2022, 71, 1399–1418.

25. Chithambarathanu, M.; Jeyakumar, M.K. Survey on crop pest detection using deep learning and machine learning approaches. Multimed. Tools Appl. 2023, 11, 1–34. [CrossRef]